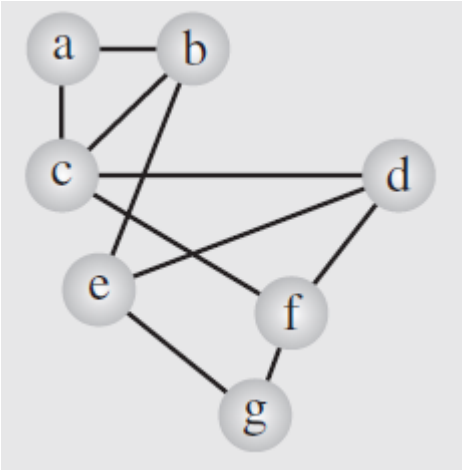
Data Structures

18. Minimum Spanning Trees

Spanning trees

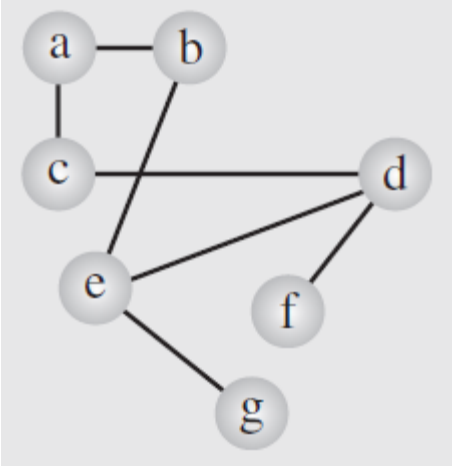
Consider the graph representing the airline connections between seven cities



If the economic situation forces this airline to shut down as many connections as possible, which of them should be retained to make sure that it is still possible to reach any city from any other city, if only indirectly?

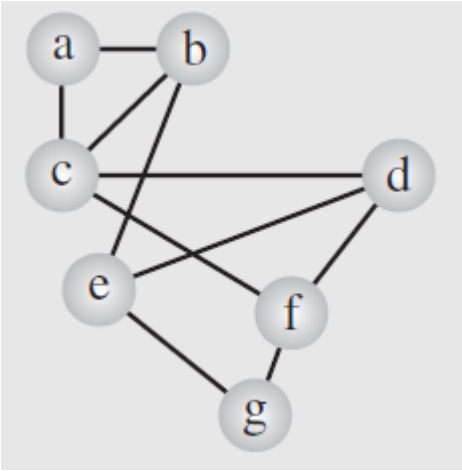
City *a* can be reached from city *d* using the path *d, c, a,* but it is also possible to use the path *d, e, b, a*

One possibility



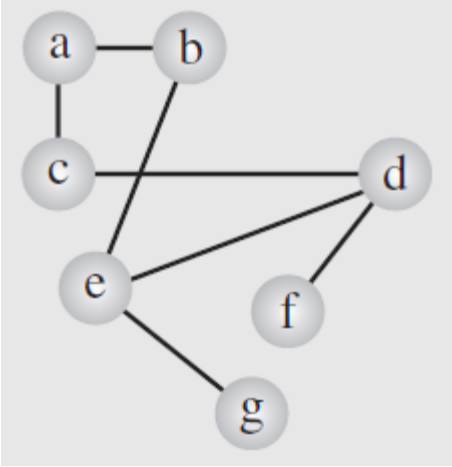
Spanning trees

Because the number of retained connections is the issue, there is still the possibility we can reduce this number



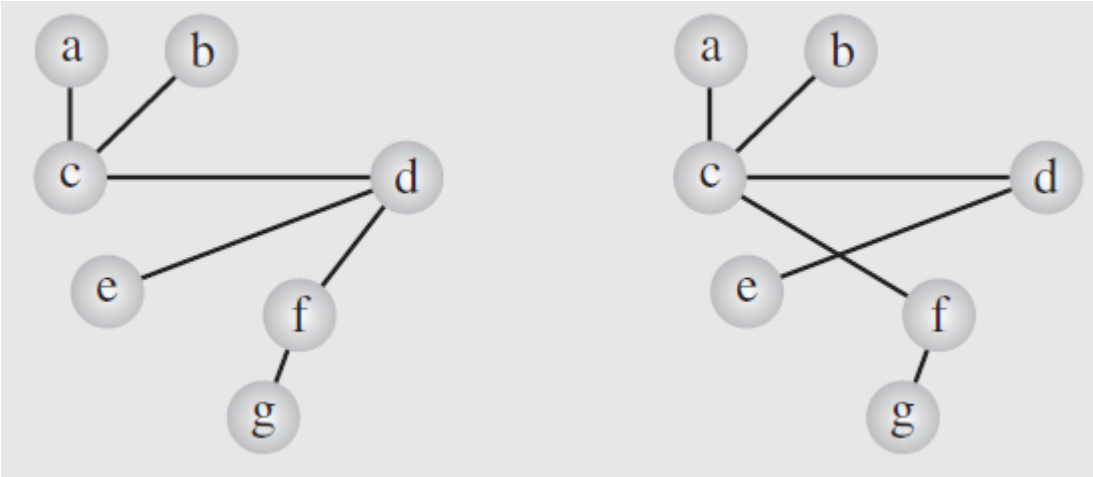
It should be clear that the minimum number of such connections form a tree because alternate paths arise as a result of cycles in the graph

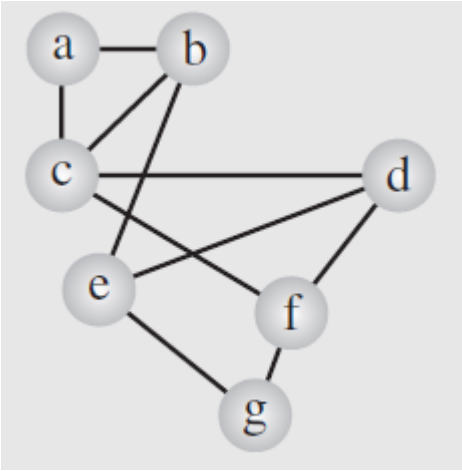
Hence, to create the minimum number of connections, a *spanning tree* should be created, and such a spanning tree is the by product of depth-first search



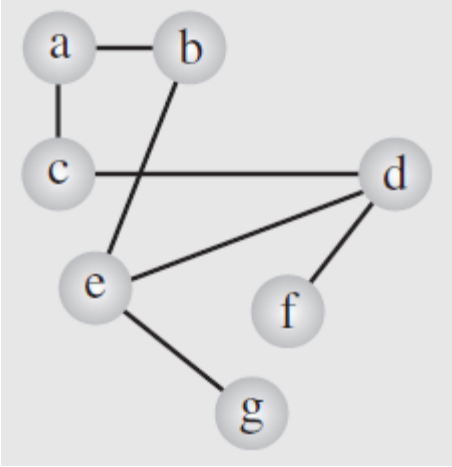
One possibility

Spanning trees





Clearly, we can create different spanning trees—that is, we can decide to retain different sets of connections—but all these trees have **six edges** and we cannot do any better than that



One possibility

Minimum spanning trees

- The solution to this problem is not optimal in that the distances between cities have not been taken into account

- Because there are alternative six-edge connections between cities, the airline uses the cost of these connections to choose the best, guaranteeing the optimum cost

- This can be achieved by having shortest distances for the six connections

- This problem can now be phrased as finding a ***minimum spanning tree***, which is a spanning tree in which the sum of the weights of its edges is minimal

Minimum spanning trees

- Given a connected graph with |V| = n vertices, a *spanning tree* is,
  - a collection of n - 1 edges connecting all n vertices

- A spanning tree is not necessarily unique

- **Weight of a spanning tree** is the sum of the weights on all the edges in the spanning tree

- **Minimum spanning trees** is a tree which minimizes this weight

Minimum spanning trees

Applications

- Lay down the circuit elements on a board
- Determine location of loading docks within a building
- Find best means to connect multiple LANs
- Determine manageable clusters of terminals within an ad hoc network
- …

Minimum spanning trees

- Many solutions exist to this minimum spanning tree problem

- Two famous algorithms
  - Prim's algorithm
  - Kruskal's algorithm

Prim's Algorithm: overview

- Input
  - An undirected weighted graph
- Basic idea
  - Select unvisited vertex with minimum distance
  - Mark it as visited for each adjacent vertex, if the weight of the connecting edge is less than its current distance
    - Update distance of the adjacent vertex
    - Set current vertex as parent of the adjacent vertex
- Halting point
  - No unvisited vertices with distance $< \infty$
  - **Note:** distance $= \infty$ means the graph is disconnected
- Output
  - A forest of minimum spanning tree on all connected sub-graphs

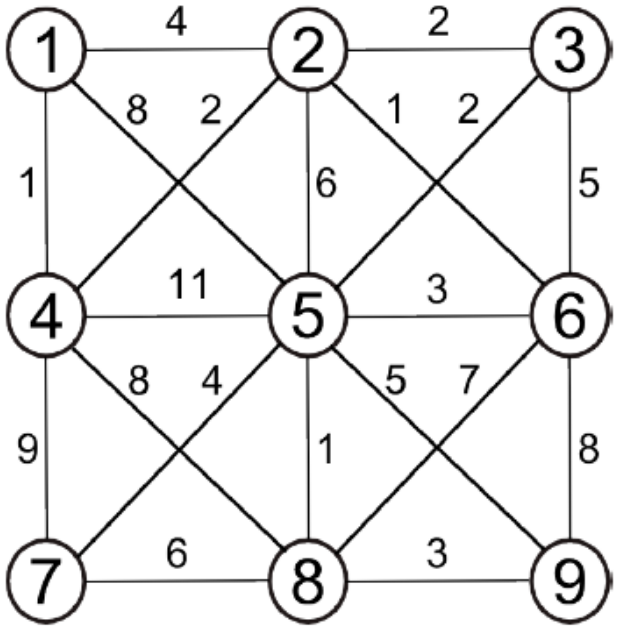
Prim's Algorithm: pseudocode

```
MST-PRIM(G, w, r)
for each u ∈ G.V
    u.key = ∞
    u.π = NIL
r.key = 0
Q = G.V
while Q ≠ ∅
    u = EXTRACT-MIN(Q)
    for each v ∈ G.Adj[u]
        if v ∈ Q and w(u, v) < v.key
            v.π = u
            v.key = w(u, v)
```

Prim's Algorithm: overview

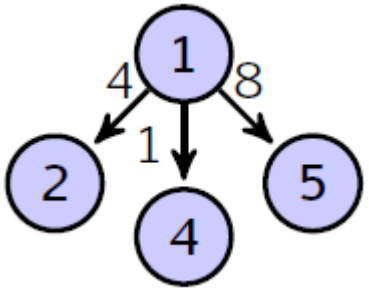
- Initialize the table
  - select root vertex, set distance to 0
  - visited to false
  - parent to 0

Vertex	Visited	Distance	Parent
1	F	0	0
2	F	∞	0
3	F	∞	0
4	F	∞	0
5	F	∞	0
6	F	∞	0
7	F	∞	0
8	F	∞	0
9	F	∞	0

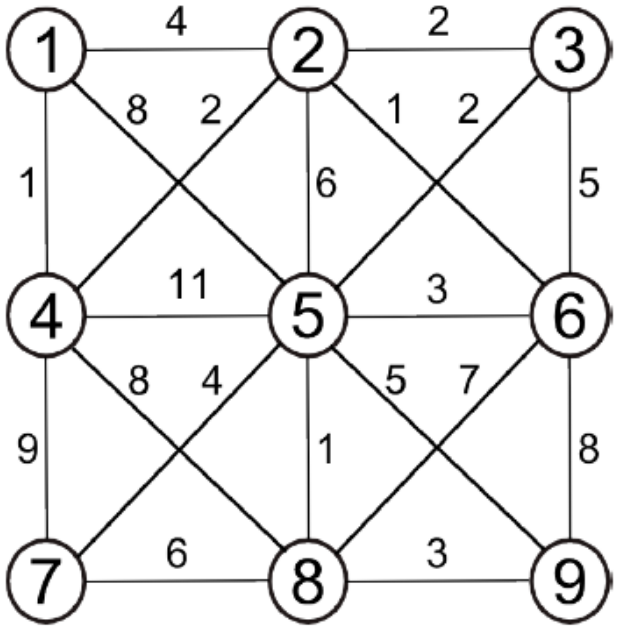


Prim's Algorithm: overview

- Visit vertex 1
  - update 2, 4 and 5
  - extend trivial tree
  - select vertex with minimum weight

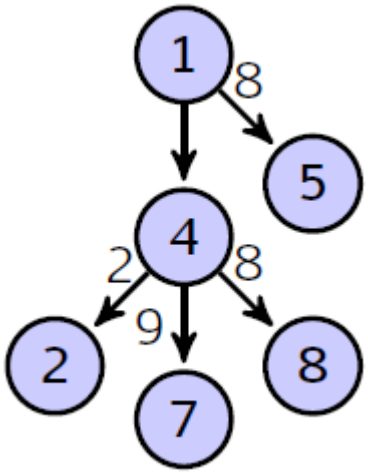


Vertex	Visited	Distance	Parent
1	T	0	0
2	F	4	1
3	F	∞	0
4	F	1	1
5	F	8	1
6	F	∞	0
7	F	∞	0
8	F	∞	0
9	F	∞	0

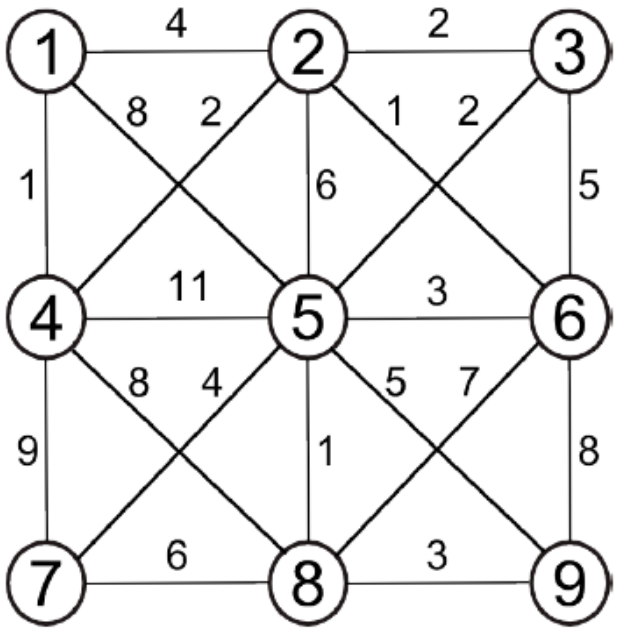


Prim's Algorithm: overview

- Visit vertex 4
  - update 2, 7 and 8, except 5
  - extend the tree
  - select vertex with minimum weight

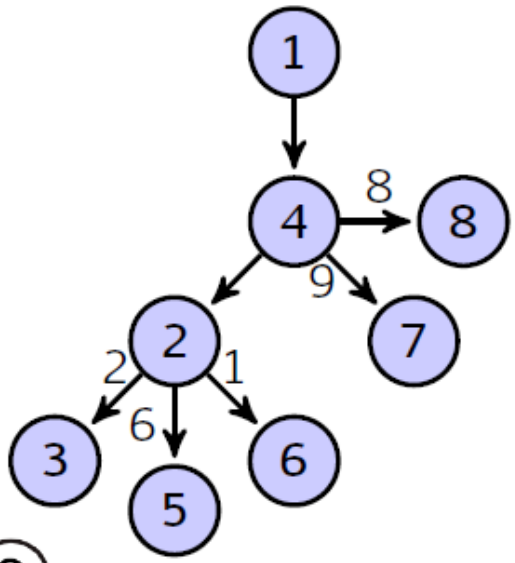


Vertex	Visited	Distance	Parent
1	T	0	0
2	F	2	4
3	F	∞	0
4	T	1	1
5	F	8	1
6	F	∞	0
7	F	9	4
8	F	8	4
9	F	∞	0

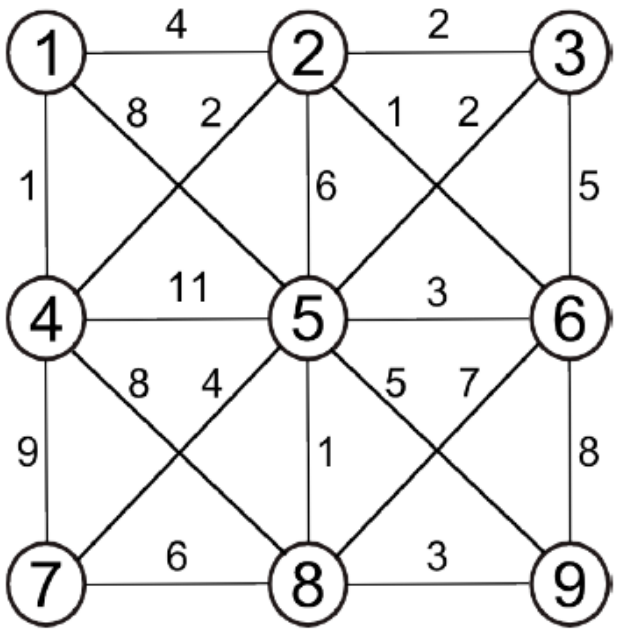


Prim's Algorithm: overview

- Visit vertex 2
  - update 3, 5 and 6
  - extend the tree
  - select vertex with minimum weight

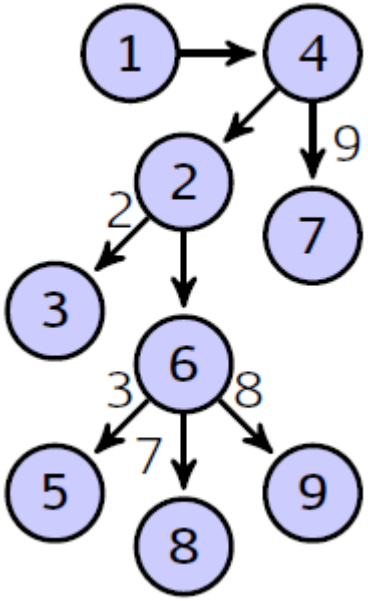


Vertex	Visited	Distance	Parent
1	T	0	0
2	T	2	4
3	F	2	2
4	T	1	1
5	F	6	2
6	F	1	2
7	F	9	4
8	F	8	4
9	F	∞	0

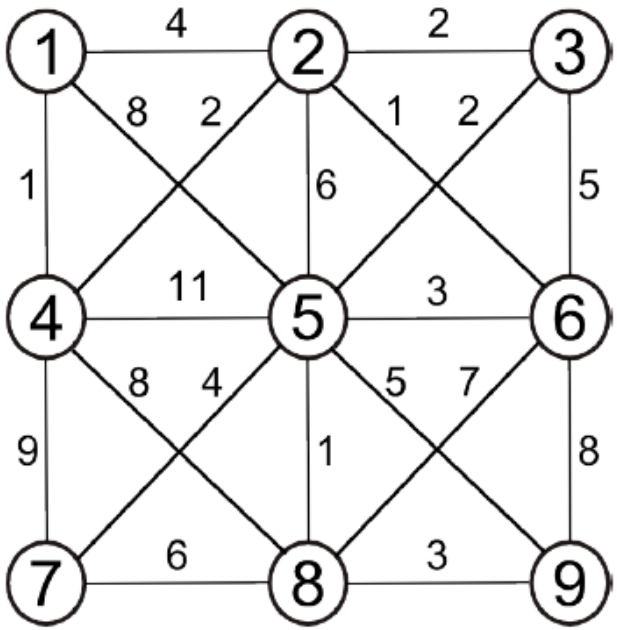


Prim's Algorithm: overview

- Visit vertex 6
  - update 5, 8 and 9, except 3
  - extend the tree
  - select vertex with minimum weight

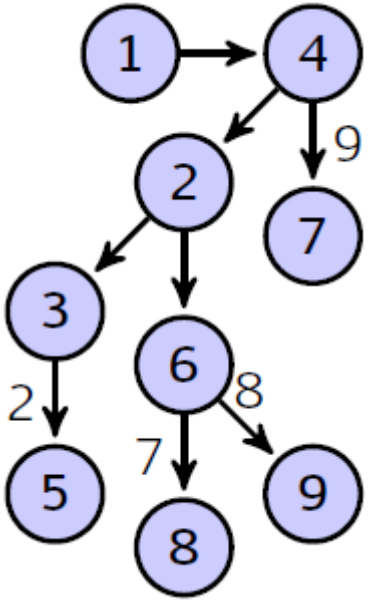


Vertex	Visited	Distance	Parent
1	T	0	0
2	T	2	4
3	F	2	2
4	T	1	1
5	F	3	6
6	T	1	2
7	F	9	4
8	F	7	6
9	F	8	6

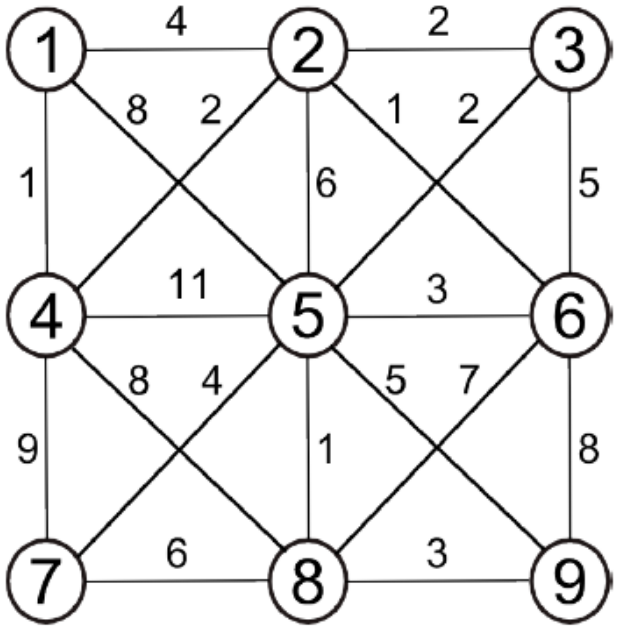


Prim's Algorithm: overview

- Visit vertex 3
  - update 5 only
  - extend the tree
  - select vertex with minimum weight



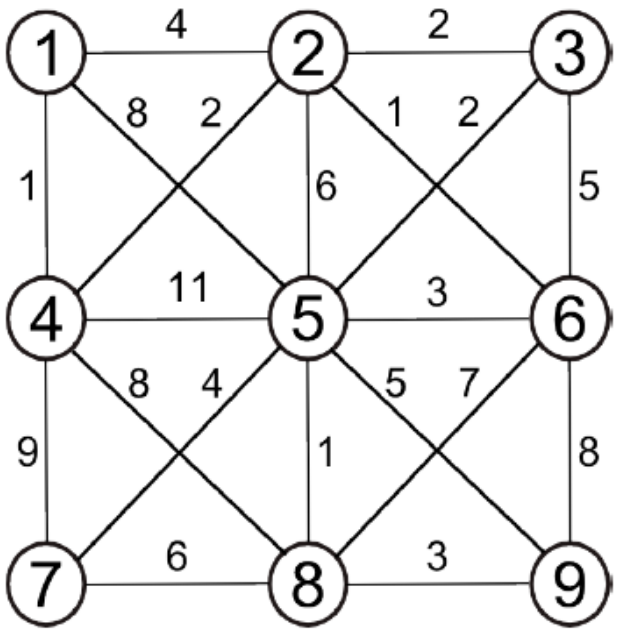
Vertex	Visited	Distance	Parent
1	T	0	0
2	T	2	4
3	T	2	2
4	T	1	1
5	F	2	3
6	T	1	2
7	F	9	4
8	F	7	6
9	F	8	6

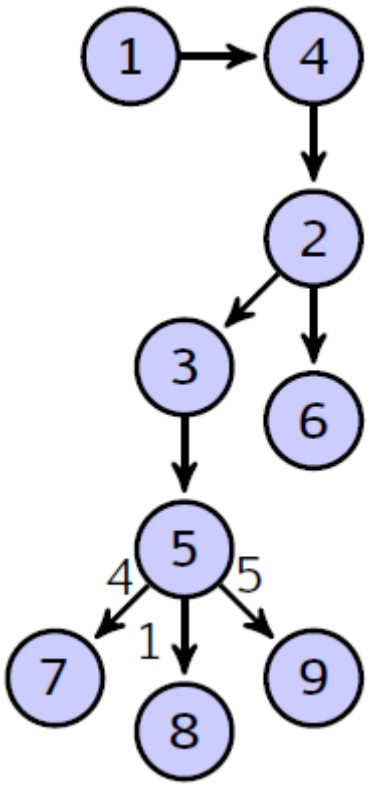


Prim's Algorithm: overview

- Visit vertex 5
  - update 7, 8 and 9
  - extend the tree
  - select vertex with minimum weight

Vertex	Visited	Distance	Parent
1	T	0	0
2	T	2	4
3	T	2	2
4	T	1	1
5	T	2	3
6	T	1	2
7	F	4	5
8	F	1	5
9	F	5	5

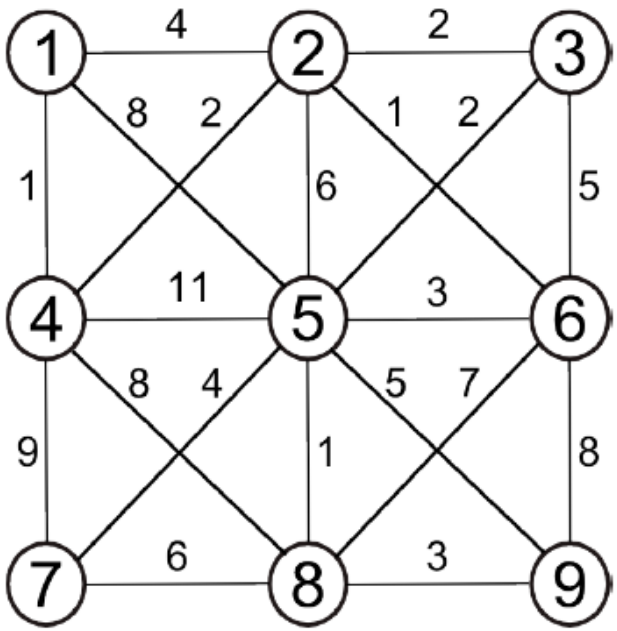


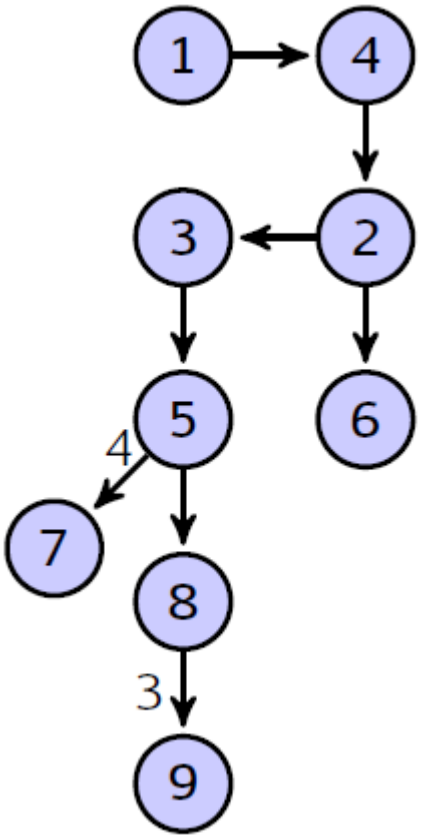


Prim's Algorithm: overview

- Visit vertex 8
  - update 9 only
  - extend the tree
  - select vertex with minimum weight

Vertex	Visited	Distance	Parent
1	T	0	0
2	T	2	4
3	T	2	2
4	T	1	1
5	T	2	3
6	T	1	2
7	F	4	5
8	T	1	5
9	F	3	8

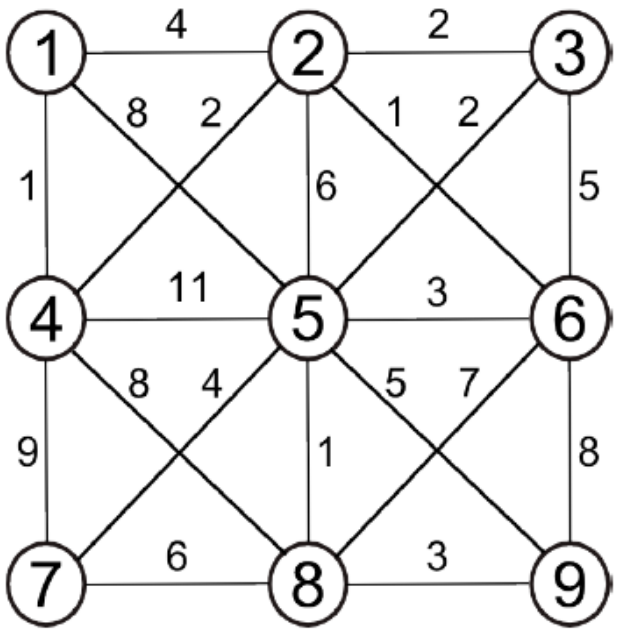


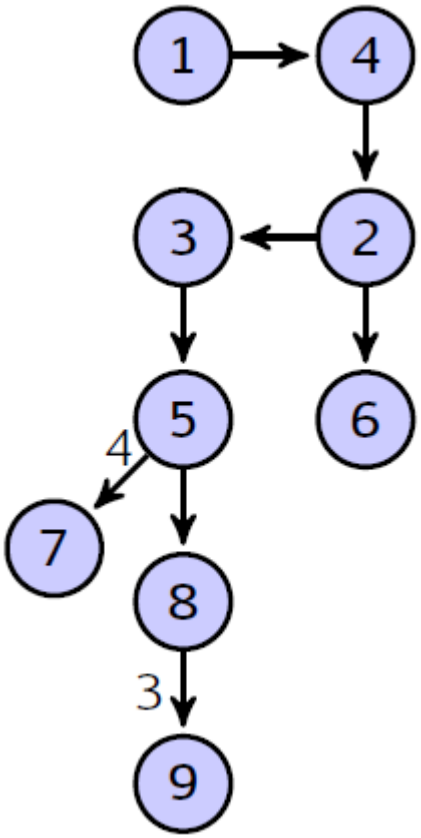


Prim's Algorithm: overview

- Visit vertex 9
  - no updates
  - no change to the tree
  - select vertex with minimum weight

Vertex	Visited	Distance	Parent
1	T	0	0
2	T	2	4
3	T	2	2
4	T	1	1
5	T	2	3
6	T	1	2
7	F	4	5
8	T	1	5
9	T	3	8

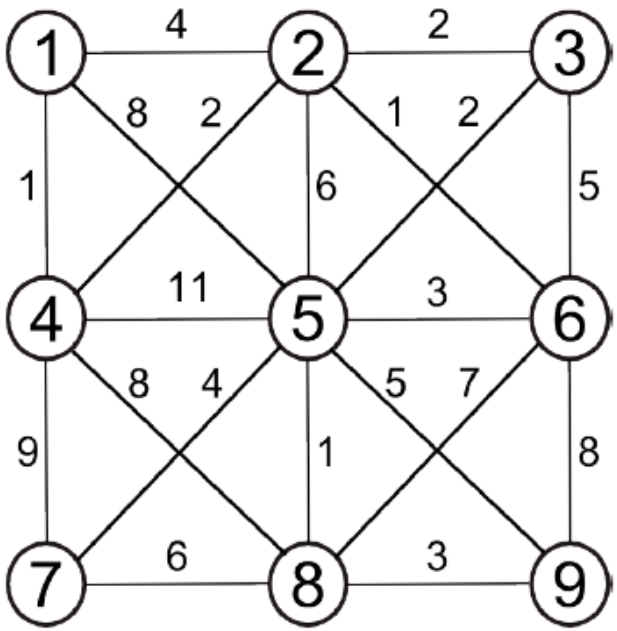


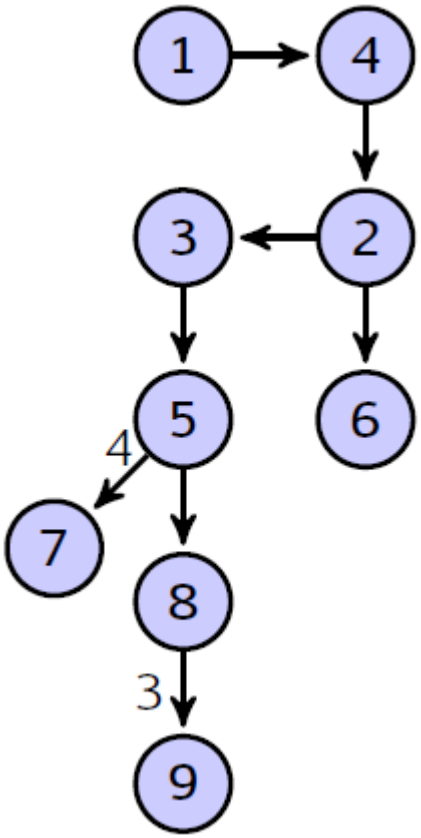


Prim's Algorithm: overview

- Visit vertex 7
  - no updates
  - no change to the tree
  - halt, as all vertices are visited

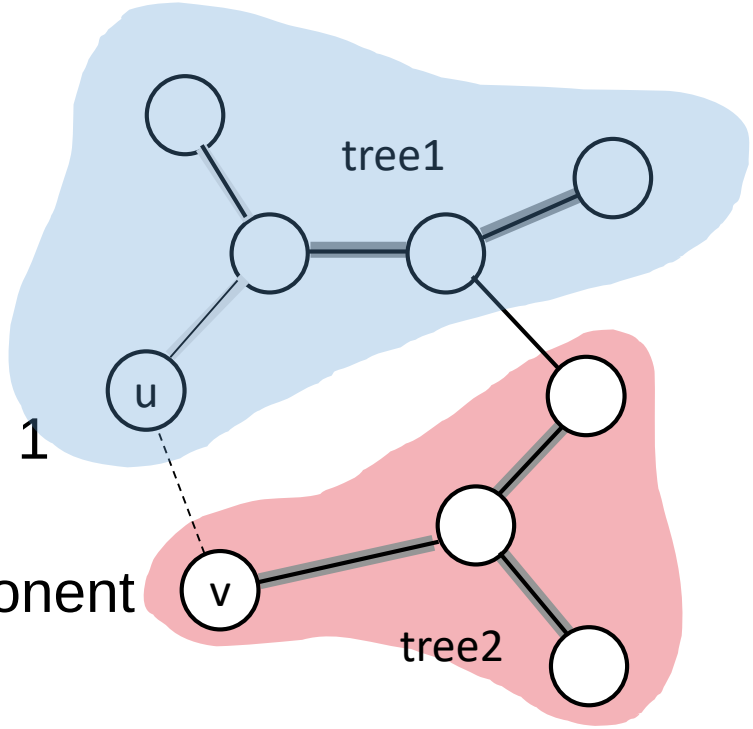
Vertex	Visited	Distance	Parent
1	T	0	0
2	T	2	4
3	T	2	2
4	T	1	1
5	T	2	3
6	T	1	2
7	T	4	5
8	T	1	5
9	T	3	8





Kruskal's Algorithm

- How is it different from Prim's algorithm?
  - Prim's algorithm grows one tree all the time
  - Kruskal's algorithm grows multiple trees (i.e., a forest) at the same time.
  - Trees are merged together using **safe** edges
  - Since an MST has exactly |V| - 1 edges, after |V| - 1 merges, we would have only one component



Kruskal's Algorithm

- Kruskal's algorithm creates a forest of trees
- Initially forest consists of N single node trees (and no edges)
- Sorts the edges by weight and goes through the edges from least weight to greatest weight
- At each step one edge (with least weight) is added so that it joins two trees together
  - As long as the addition does not create a cycle

Kruskal's Algorithm

The halting conditions are as follows:

1. When $|V| - 1$ edges have been added
   - In this case we have a minimum spanning tree

2. We have gone through all edges
   - A forest of minimum spanning trees on all connected sub-graphs

Kruskal's Algorithm

1. The forest is constructed – with each node in a separate tree

2. The edges are placed in a priority queue

3. Until we've added `n-1` edges (assumption: connected graph)
   1. Extract the cheapest edge from the queue

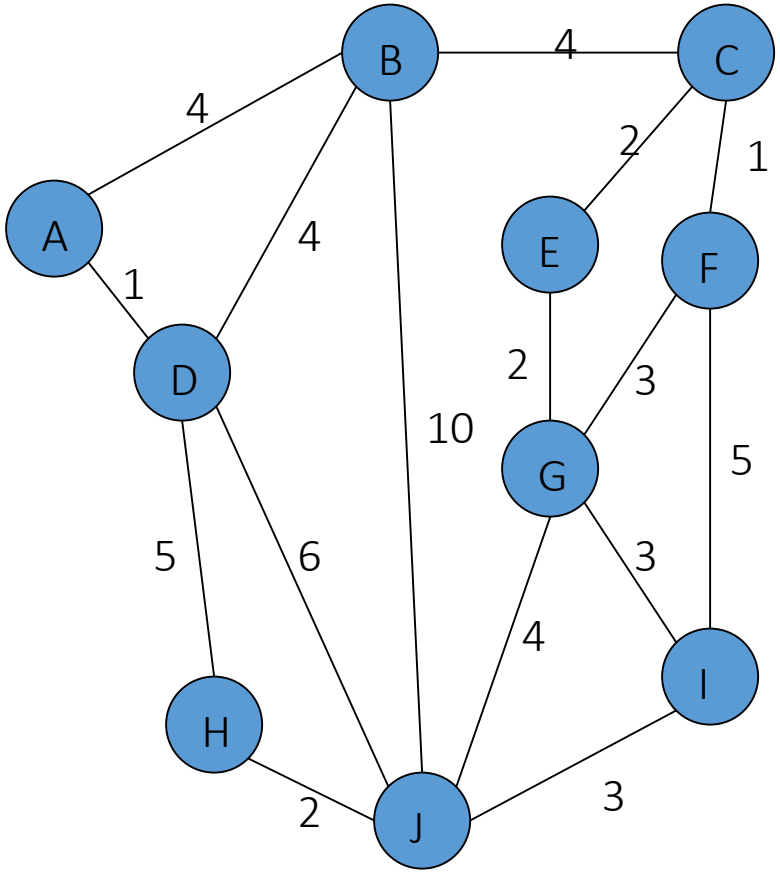
   2. If it forms a cycle, reject it

   3. Else add it to the forest. Adding it to the forest will join two trees

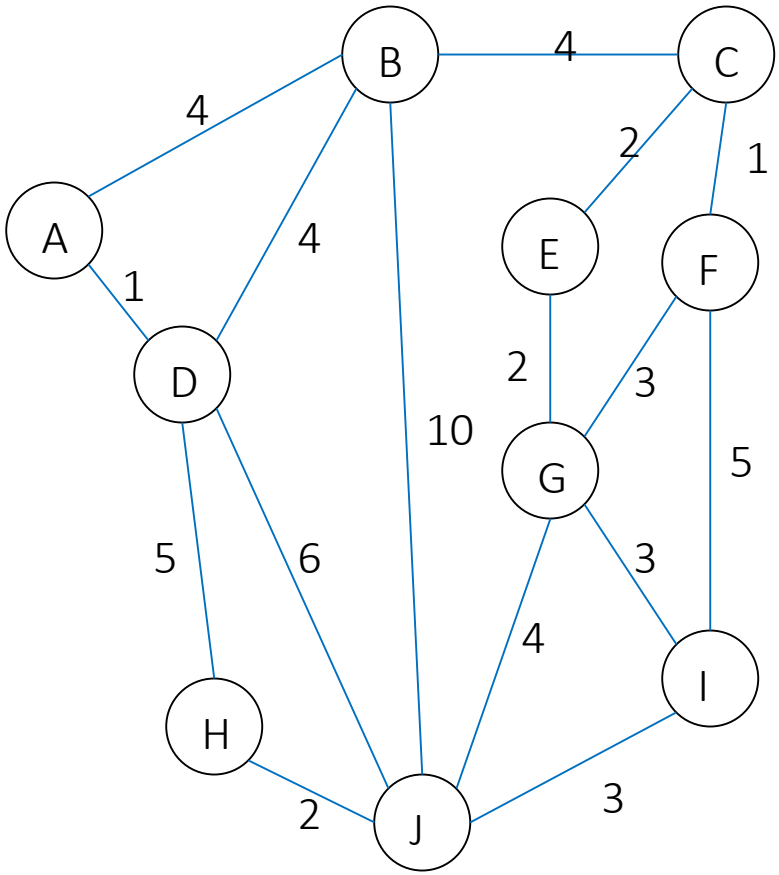
- Every step will have joined two trees in the forest together, so that at the end, there will only be one tree

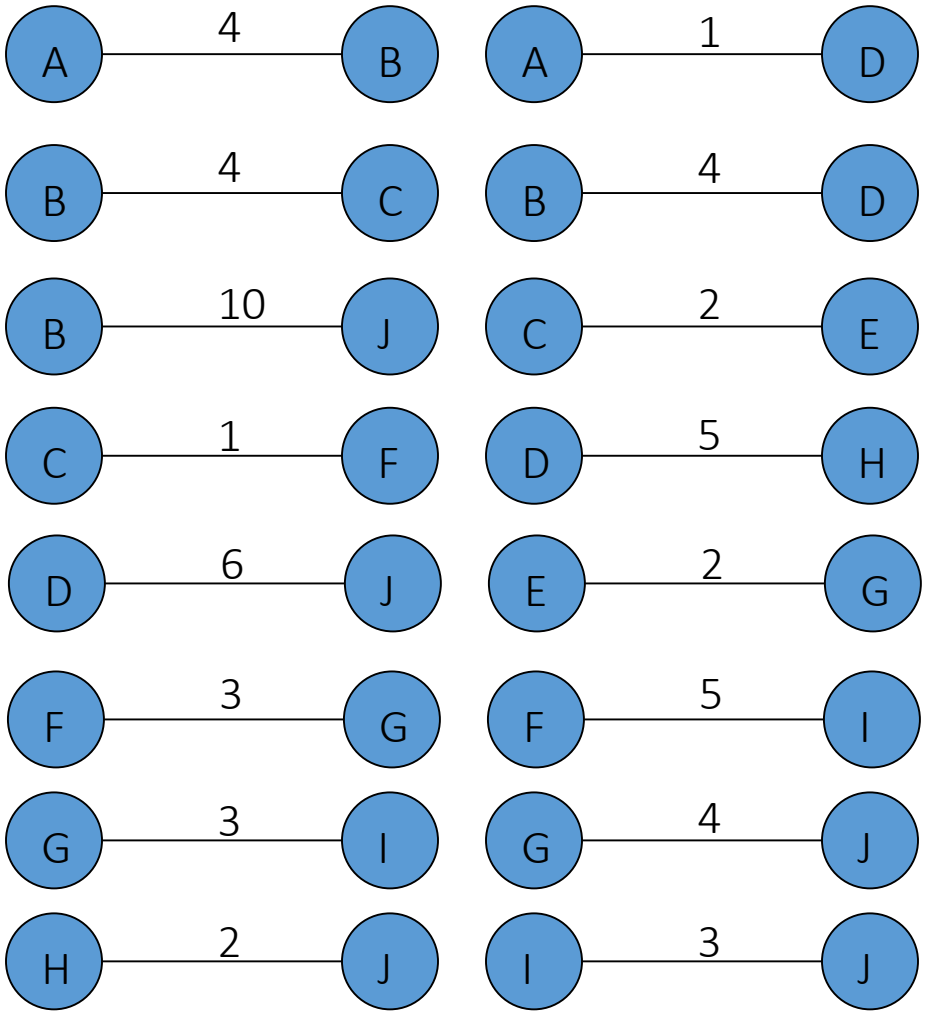
Kruskal's Algorithm – Example

- Complete Graph



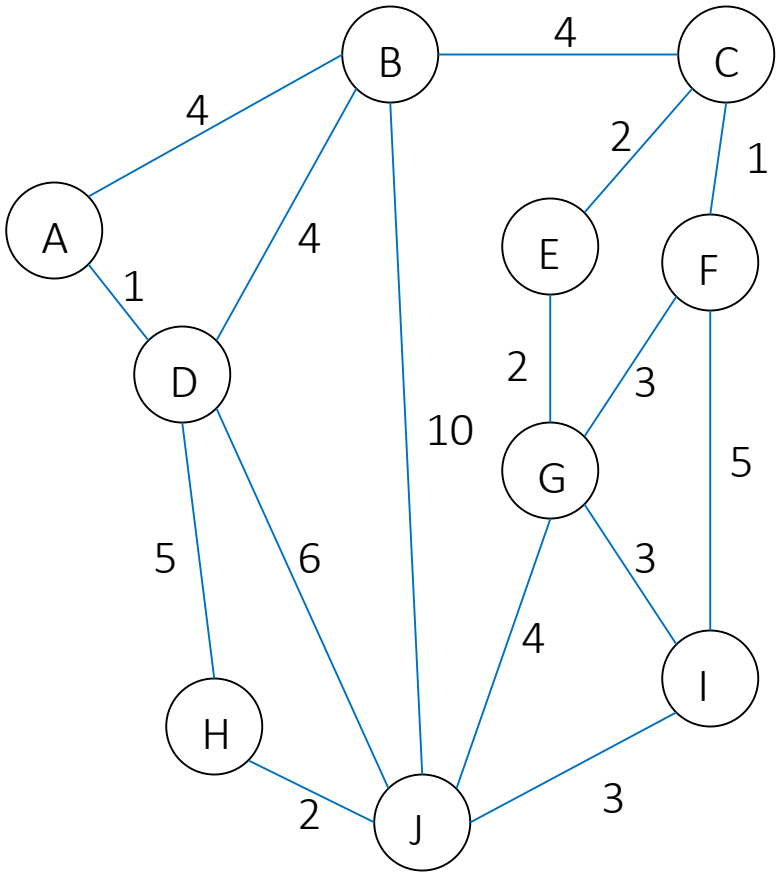
Kruskal's Algorithm – Example

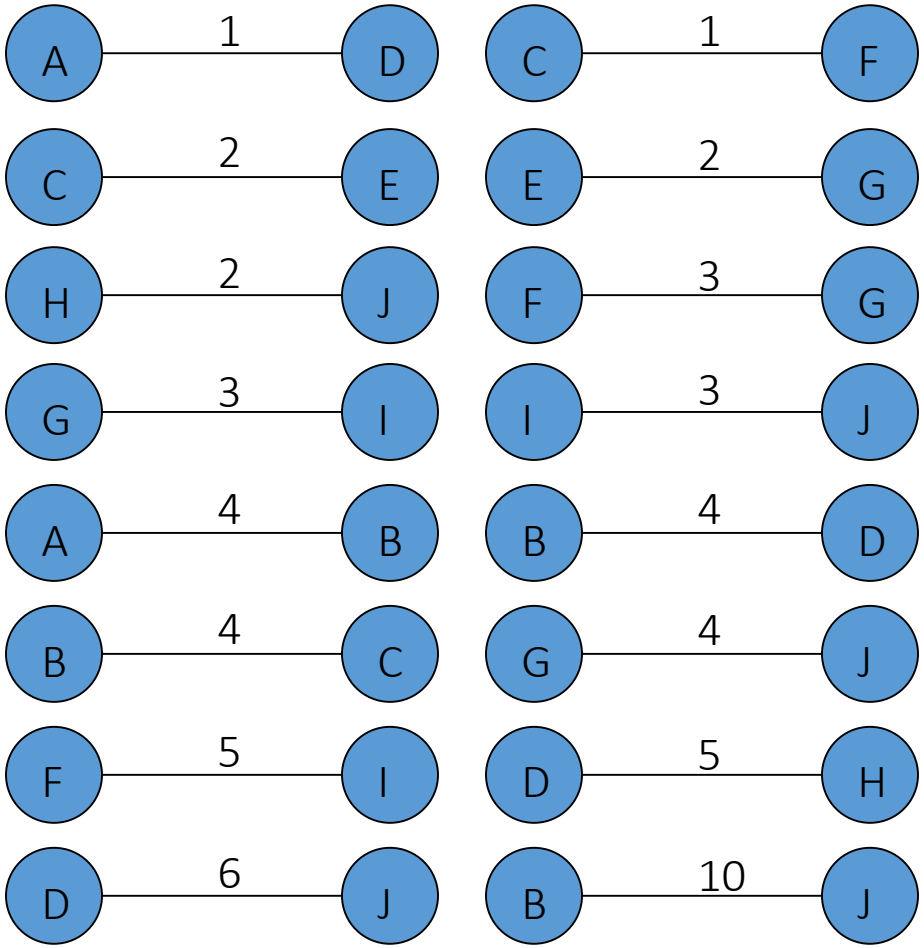




Kruskal's Algorithm – Example

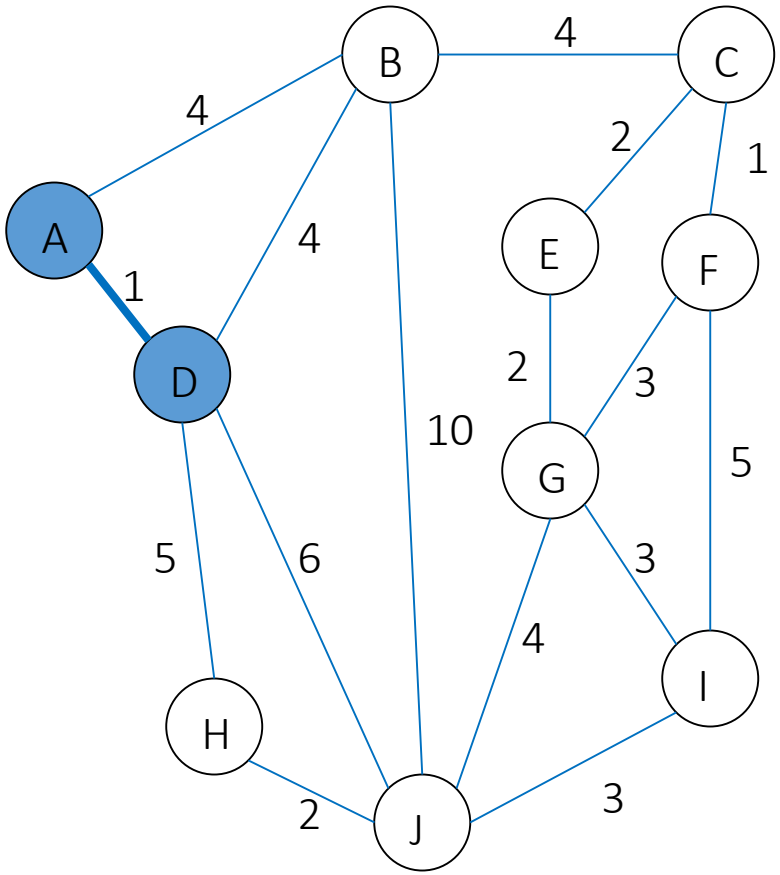
- Sort Edges: In reality priority queue is used

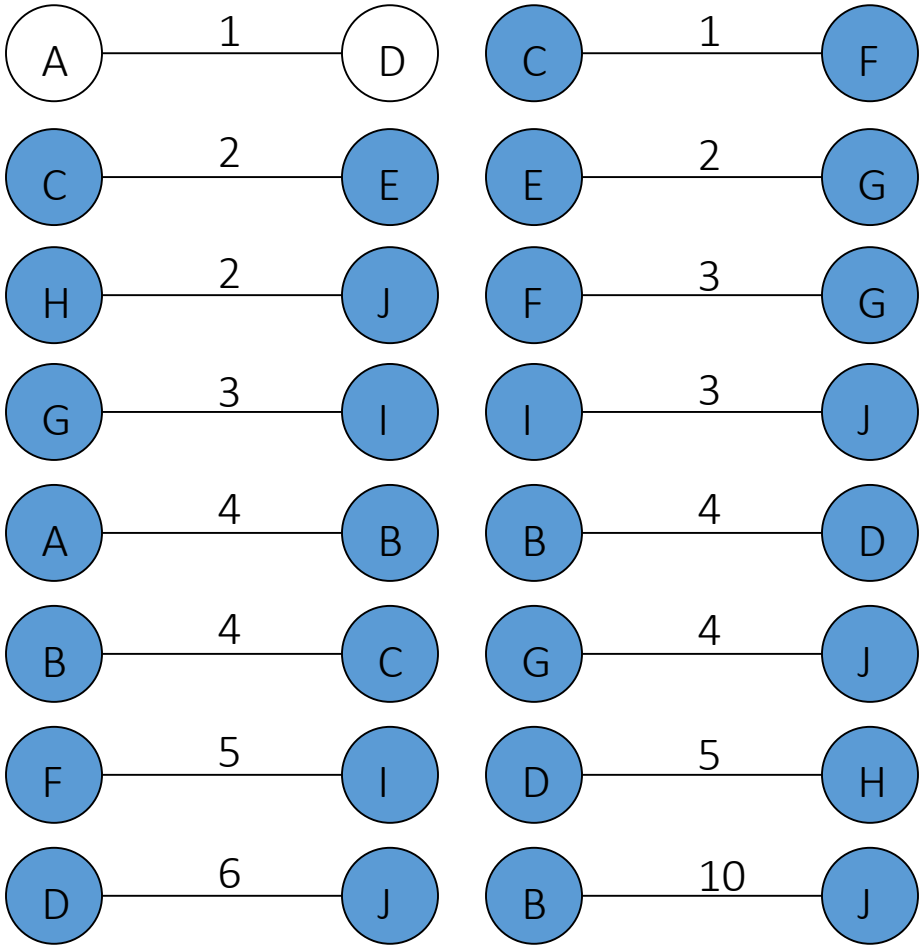




Kruskal's Algorithm – Example

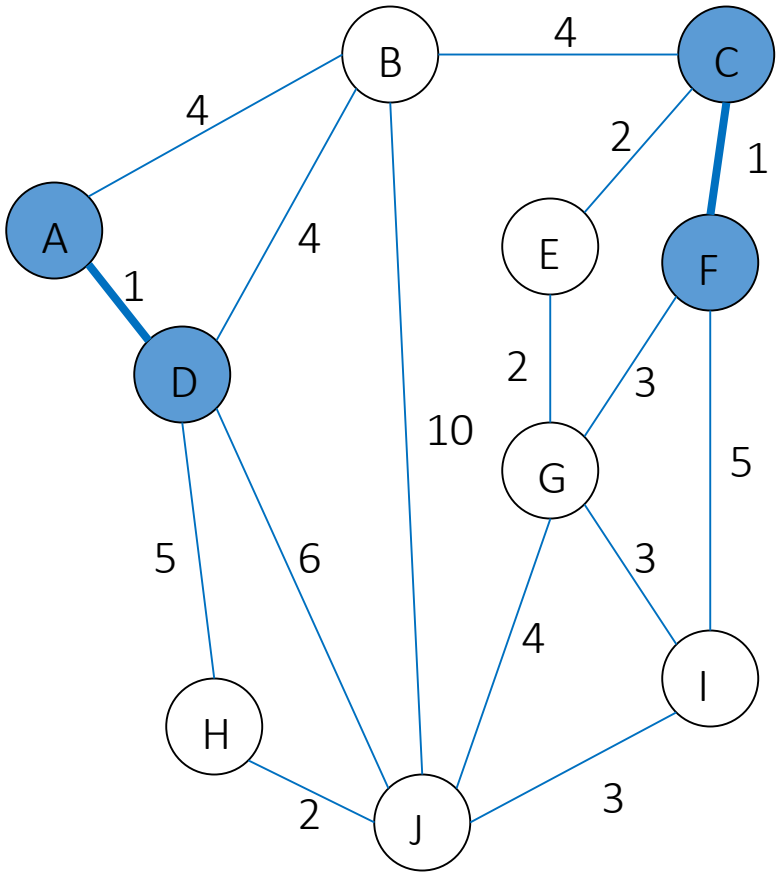
- Add Edge

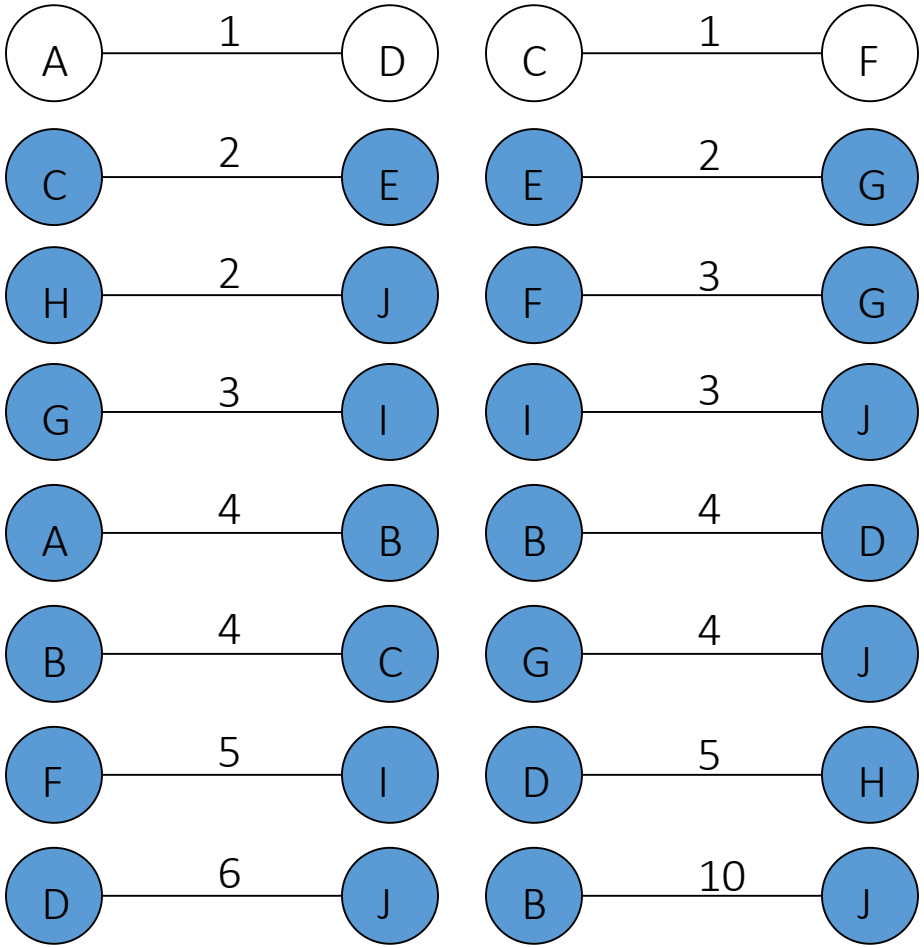




Kruskal's Algorithm – Example

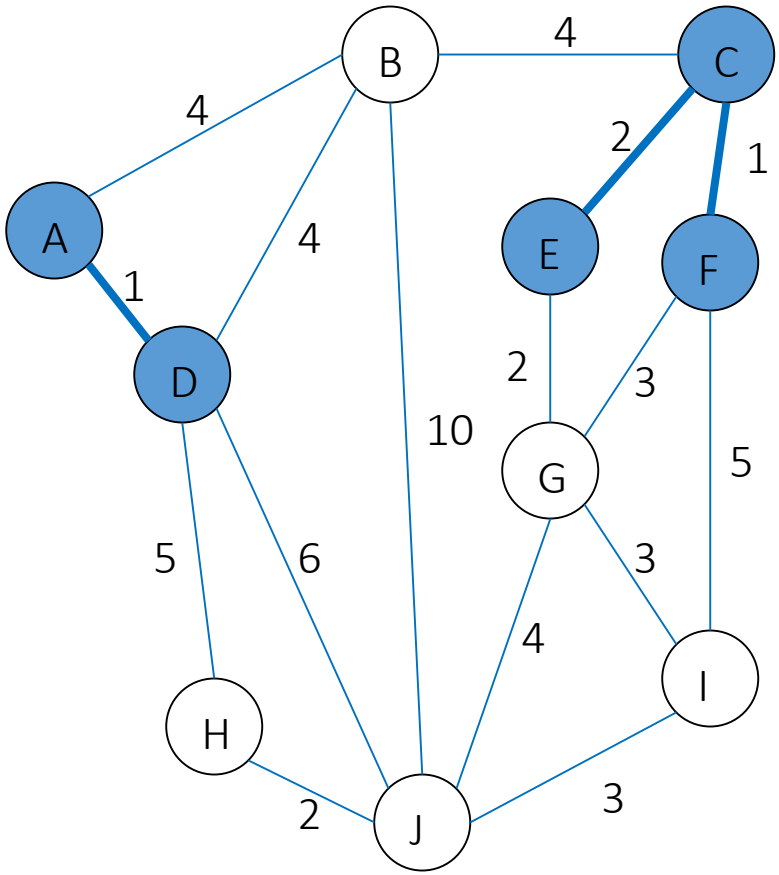
- Add Edge

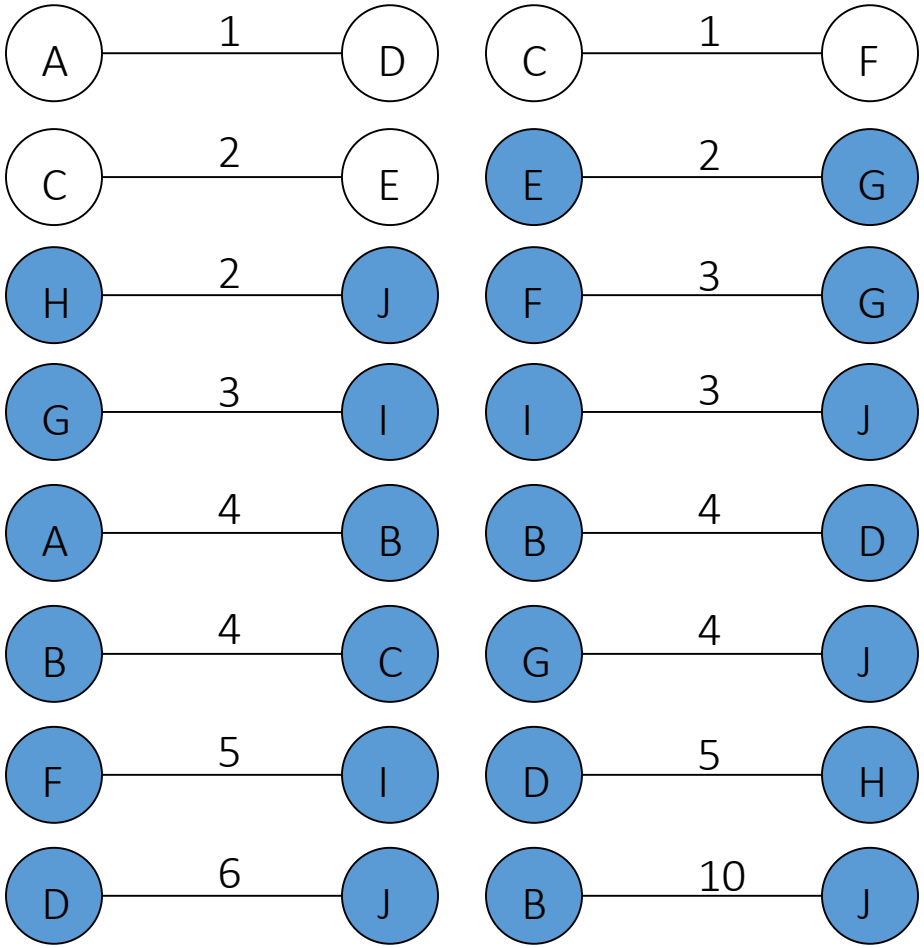




Kruskal's Algorithm – Example

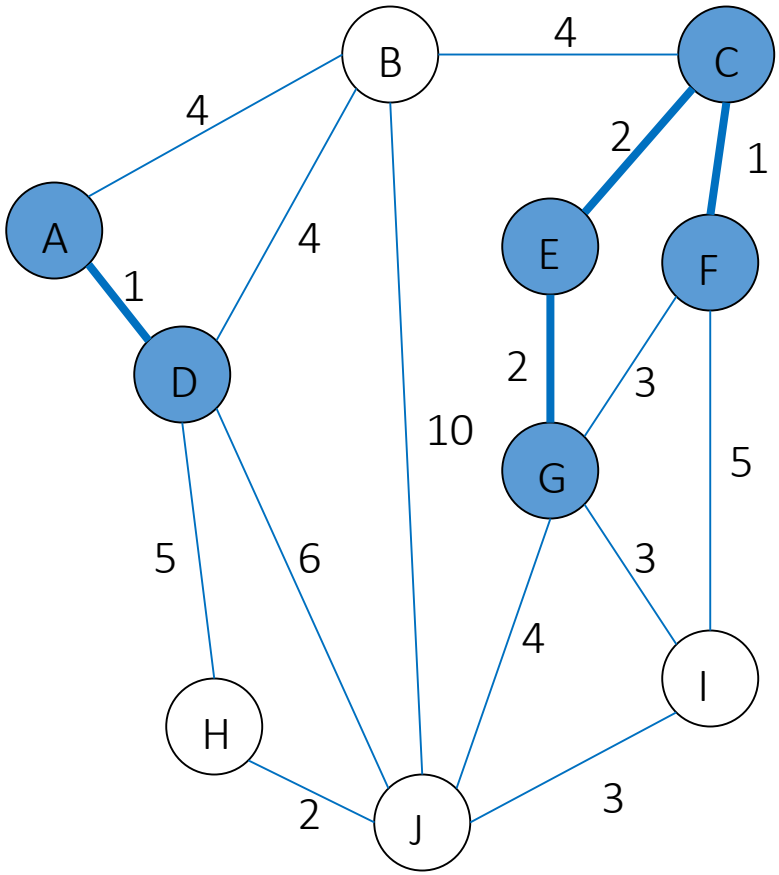
- Add Edge

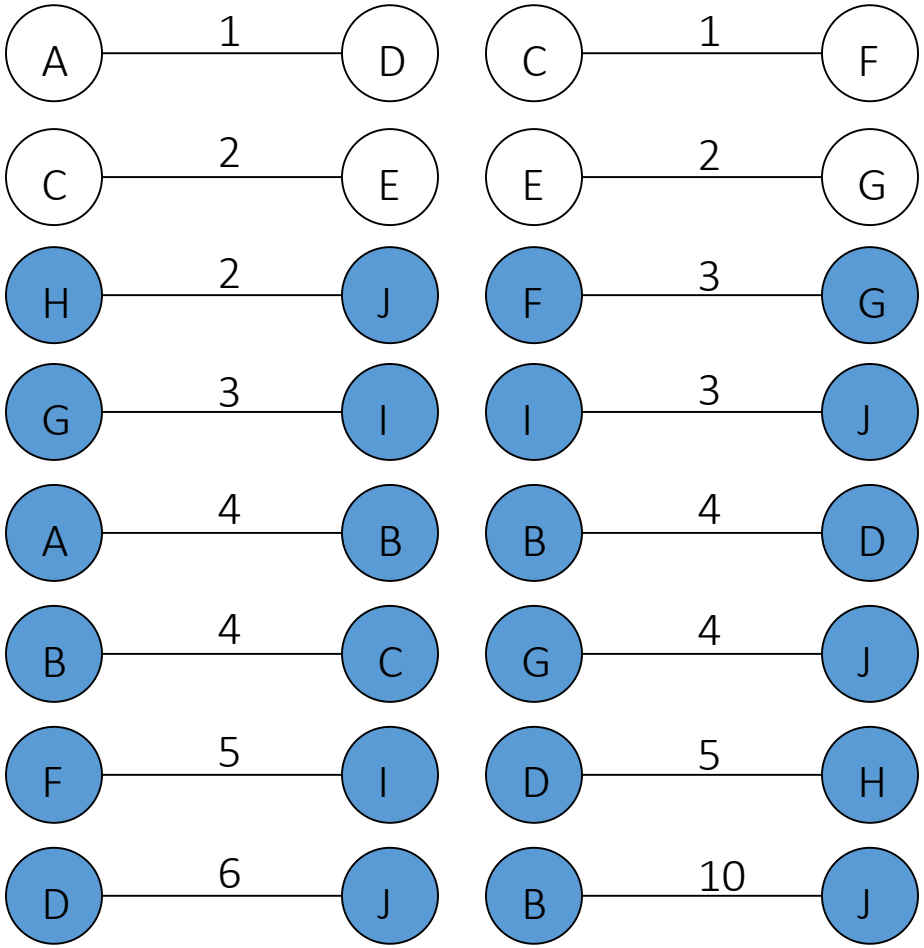




Kruskal's Algorithm – Example

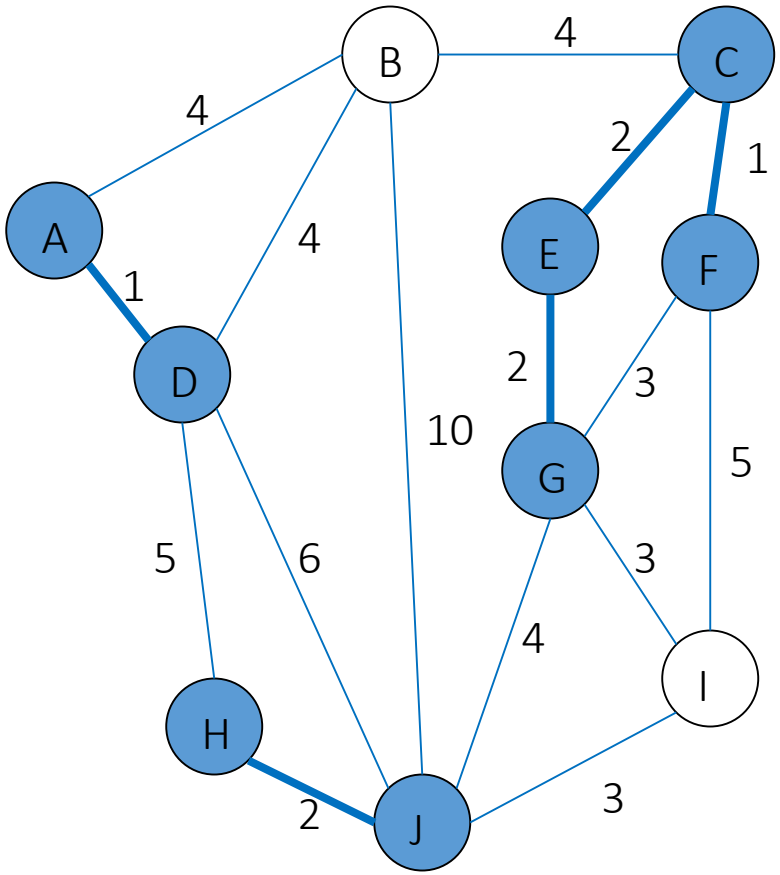
- Add Edge

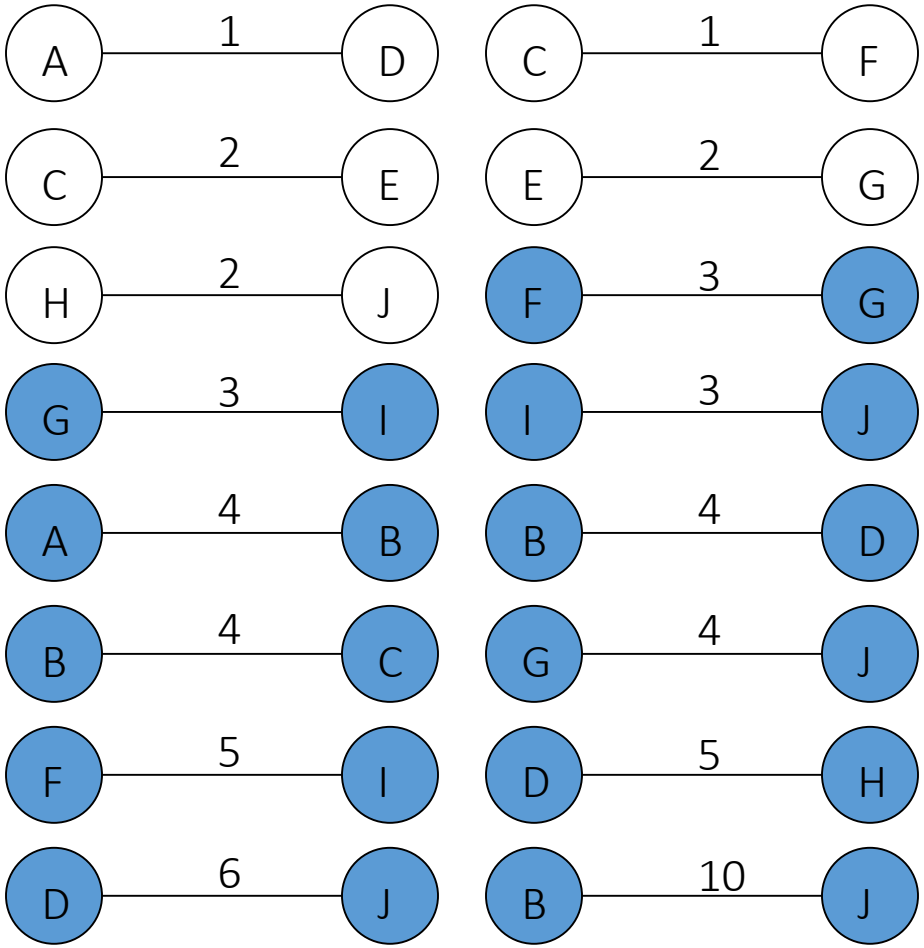




Kruskal's Algorithm – Example

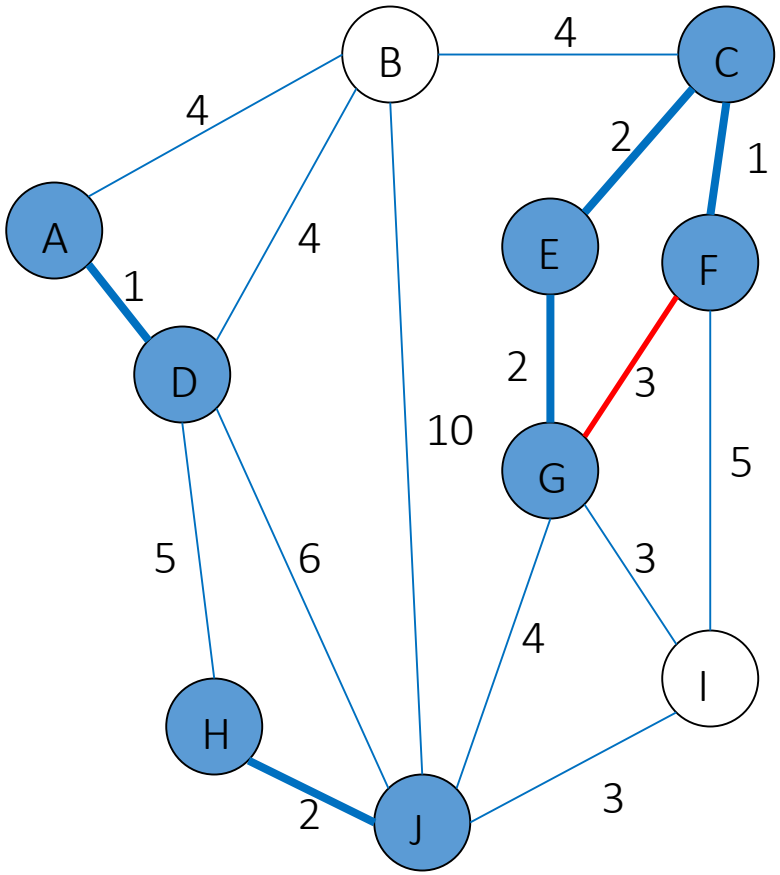
- Add Edge

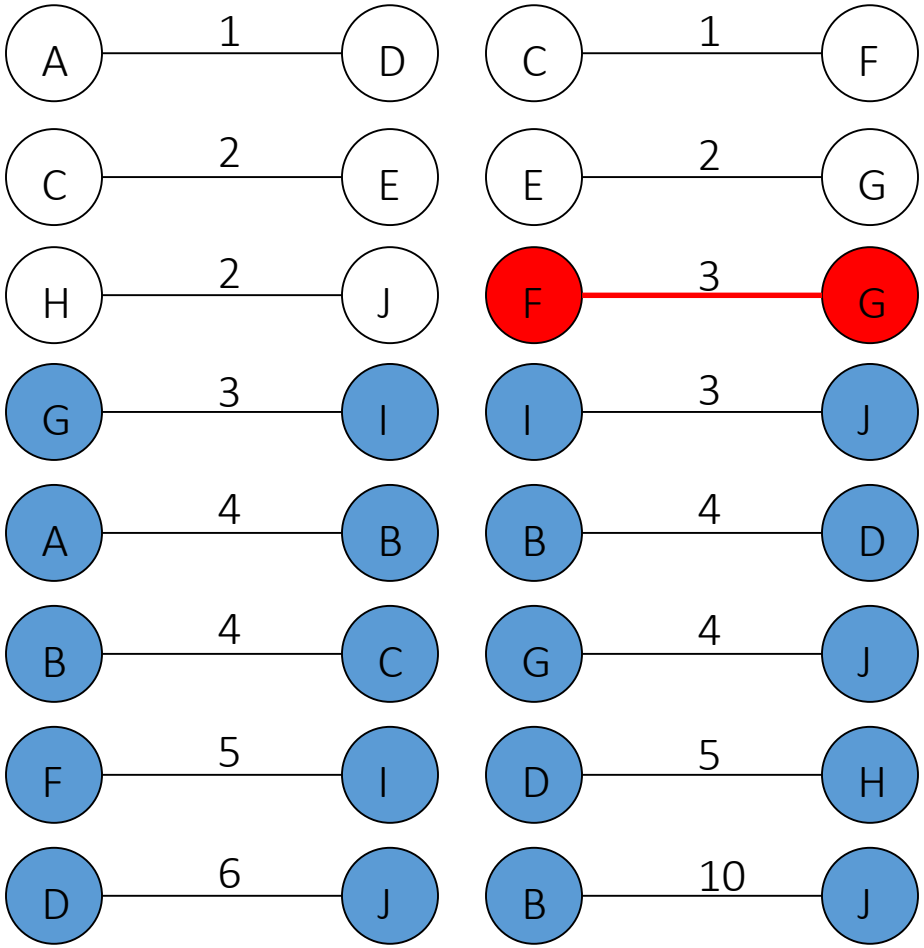




Kruskal's Algorithm – Example

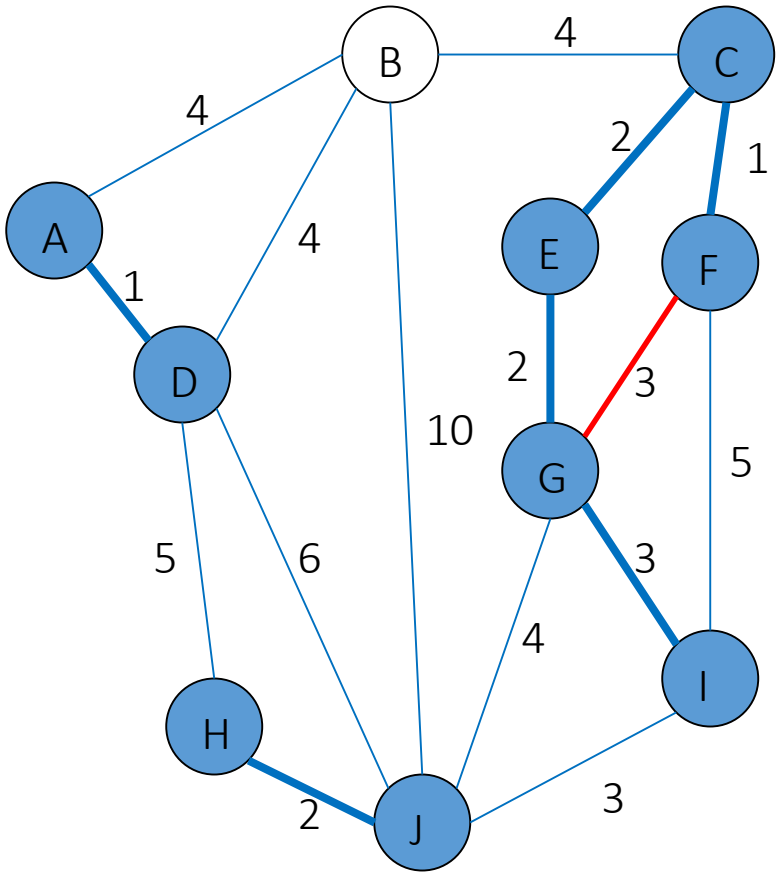
- Add Edge

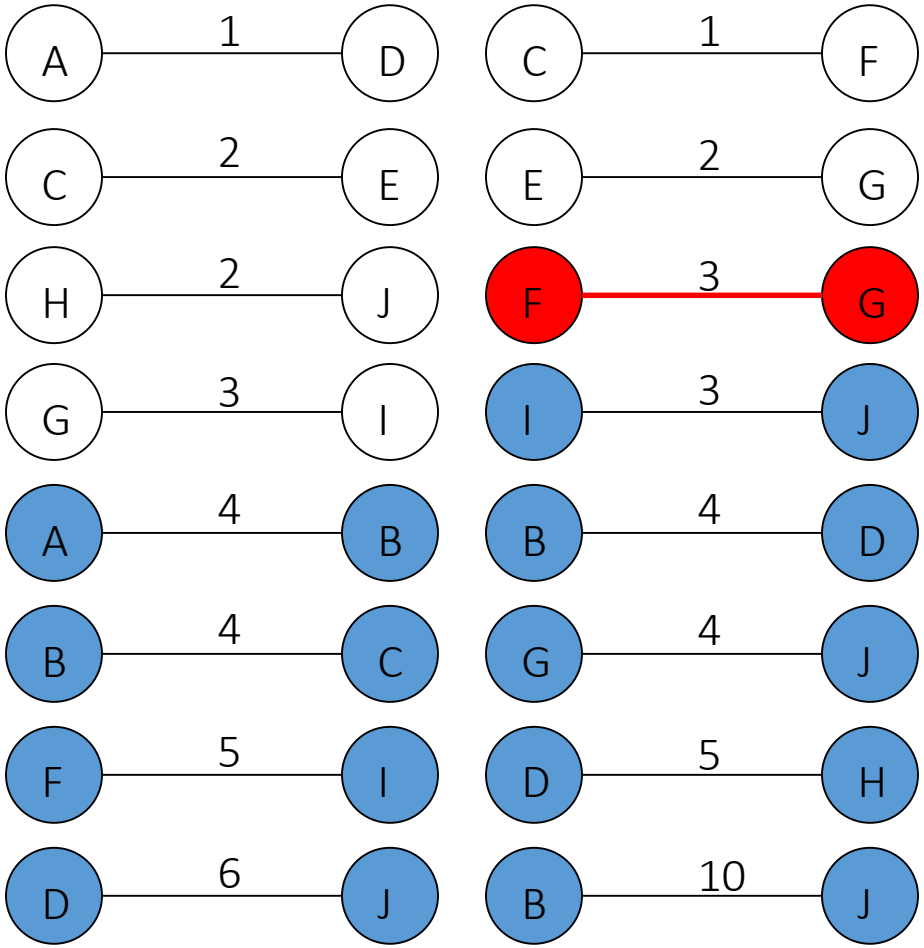




Kruskal's Algorithm – Example

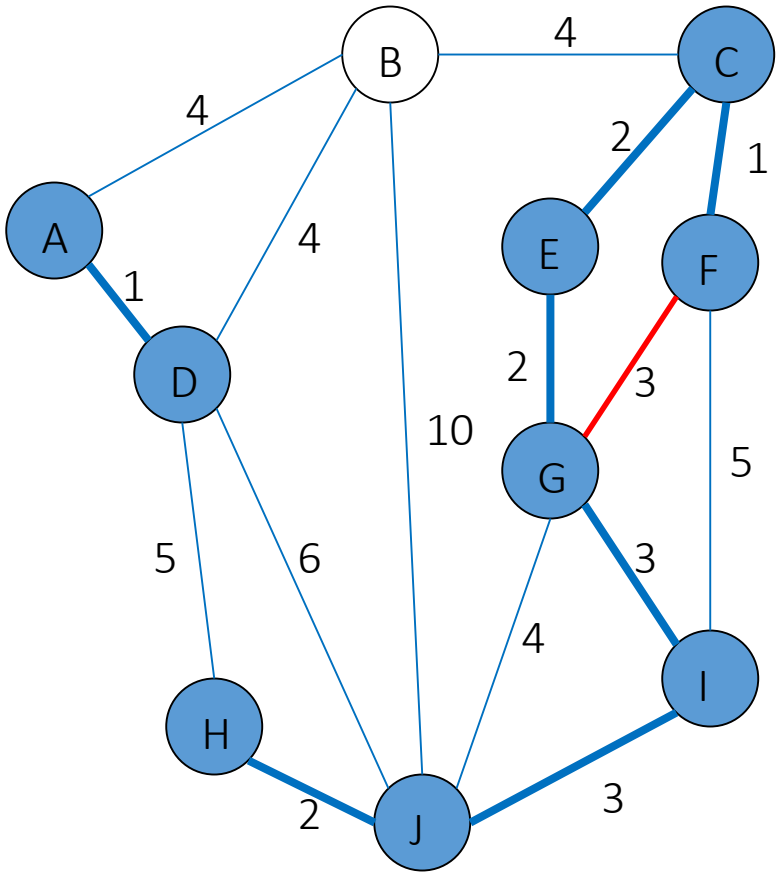
- Add Edge

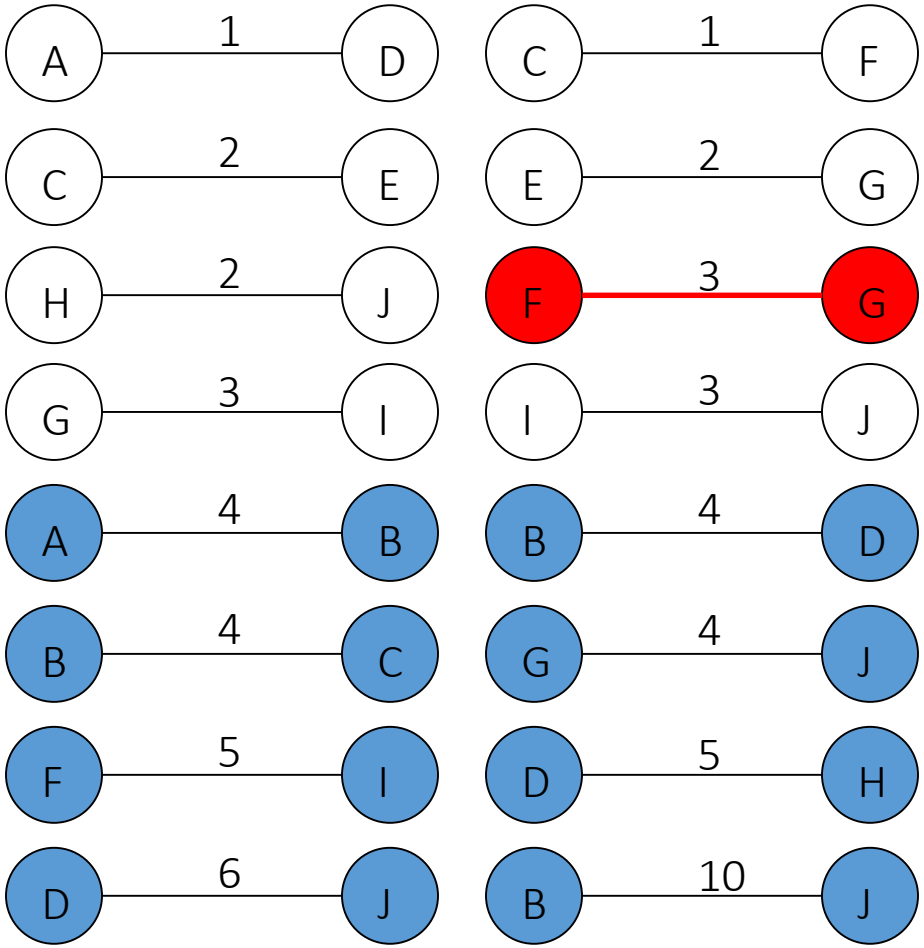




Kruskal's Algorithm – Example

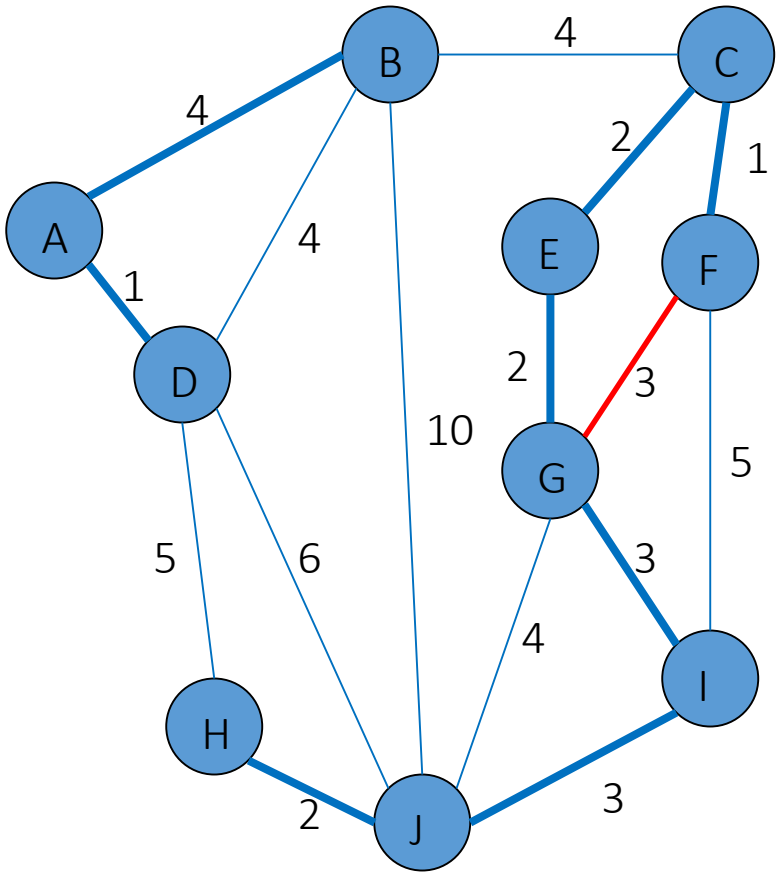
- Add Edge

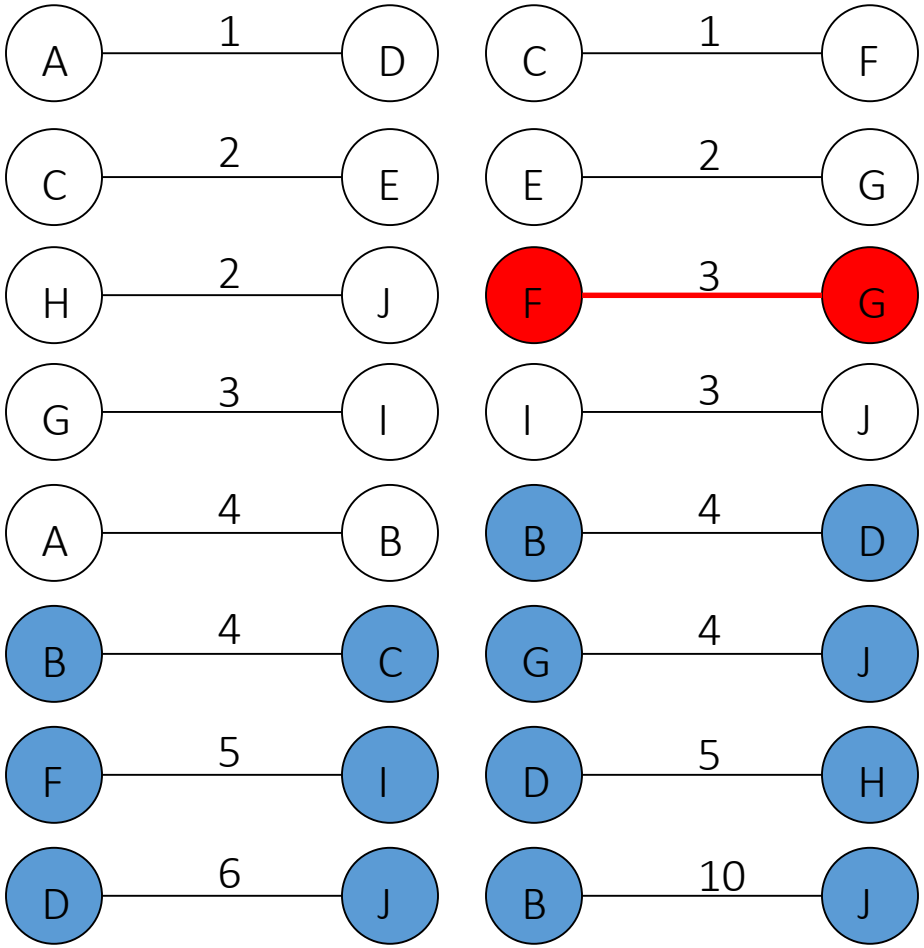




Kruskal's Algorithm – Example

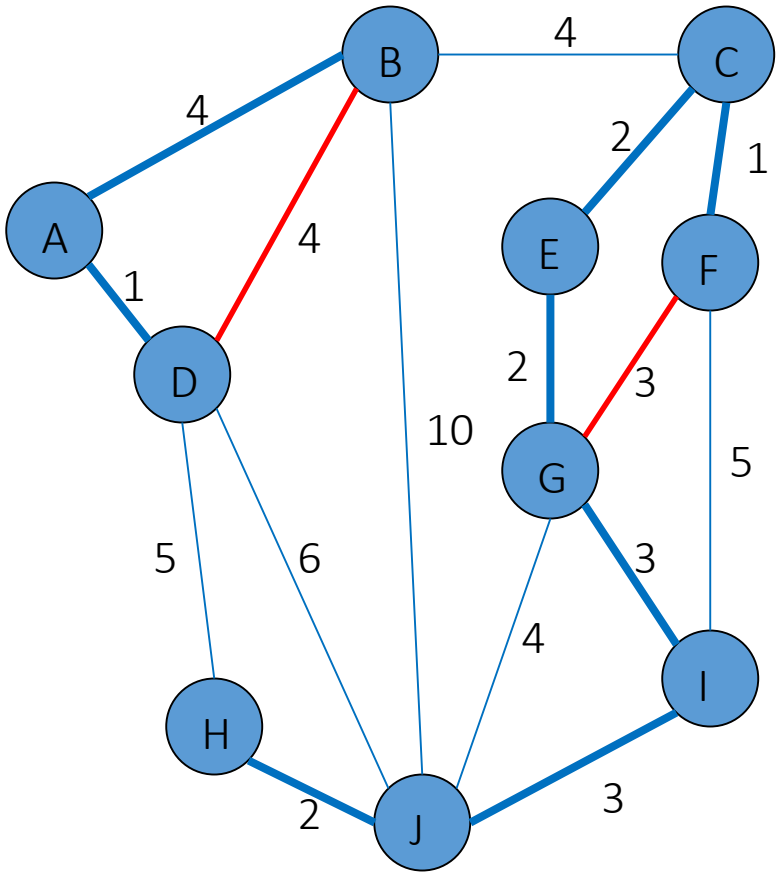
- Add Edge

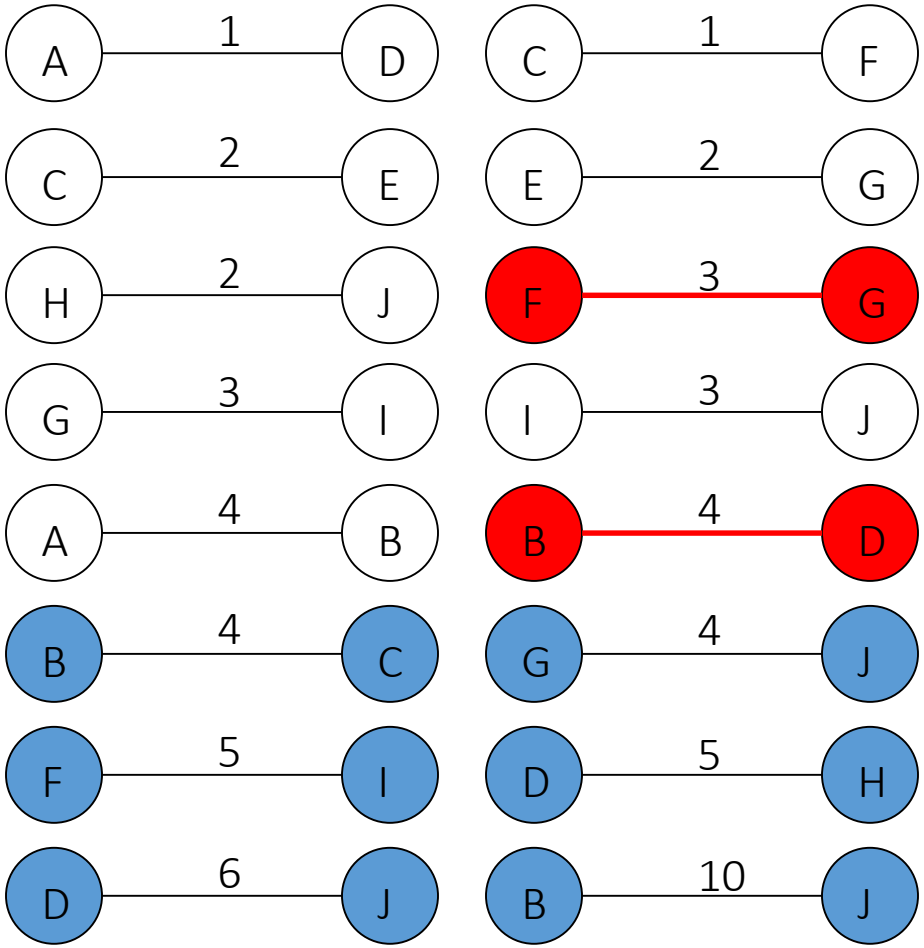




Kruskal's Algorithm – Example

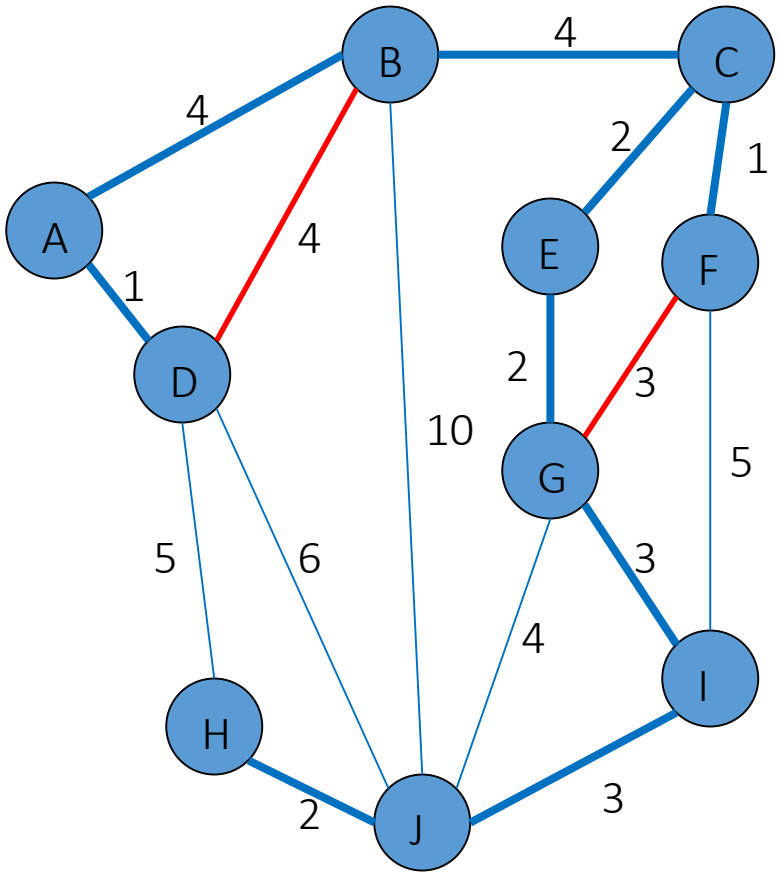
- Add Edge

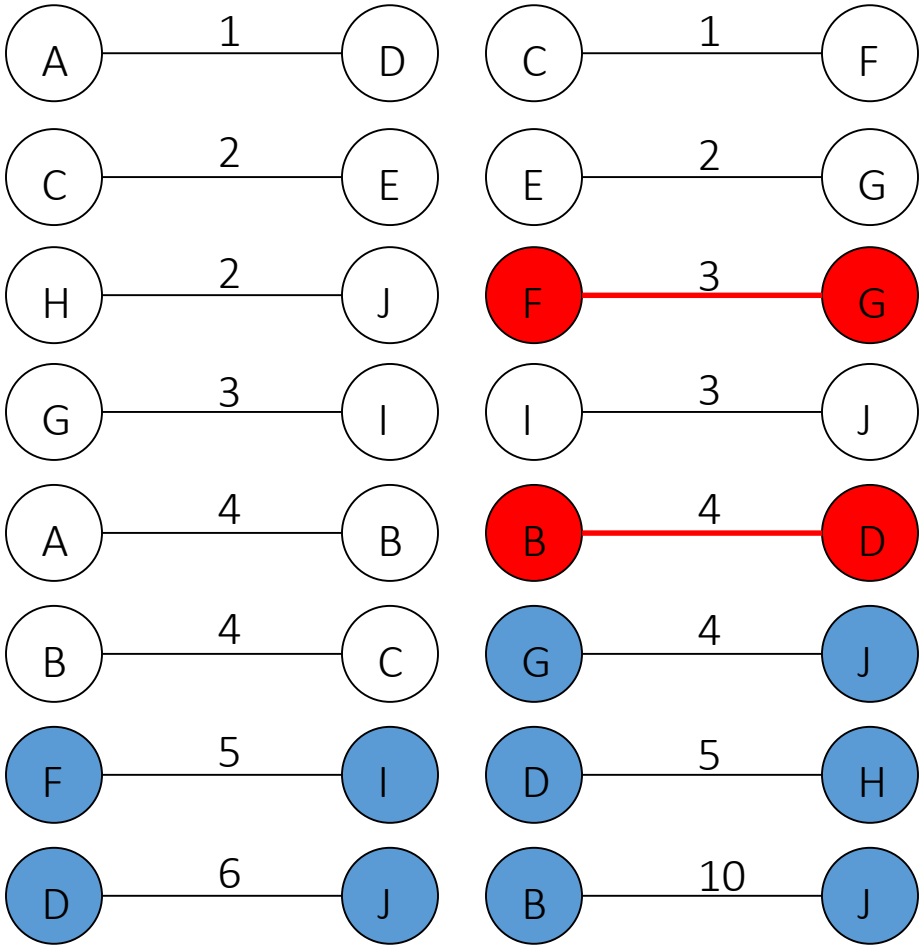




Kruskal's Algorithm – Example

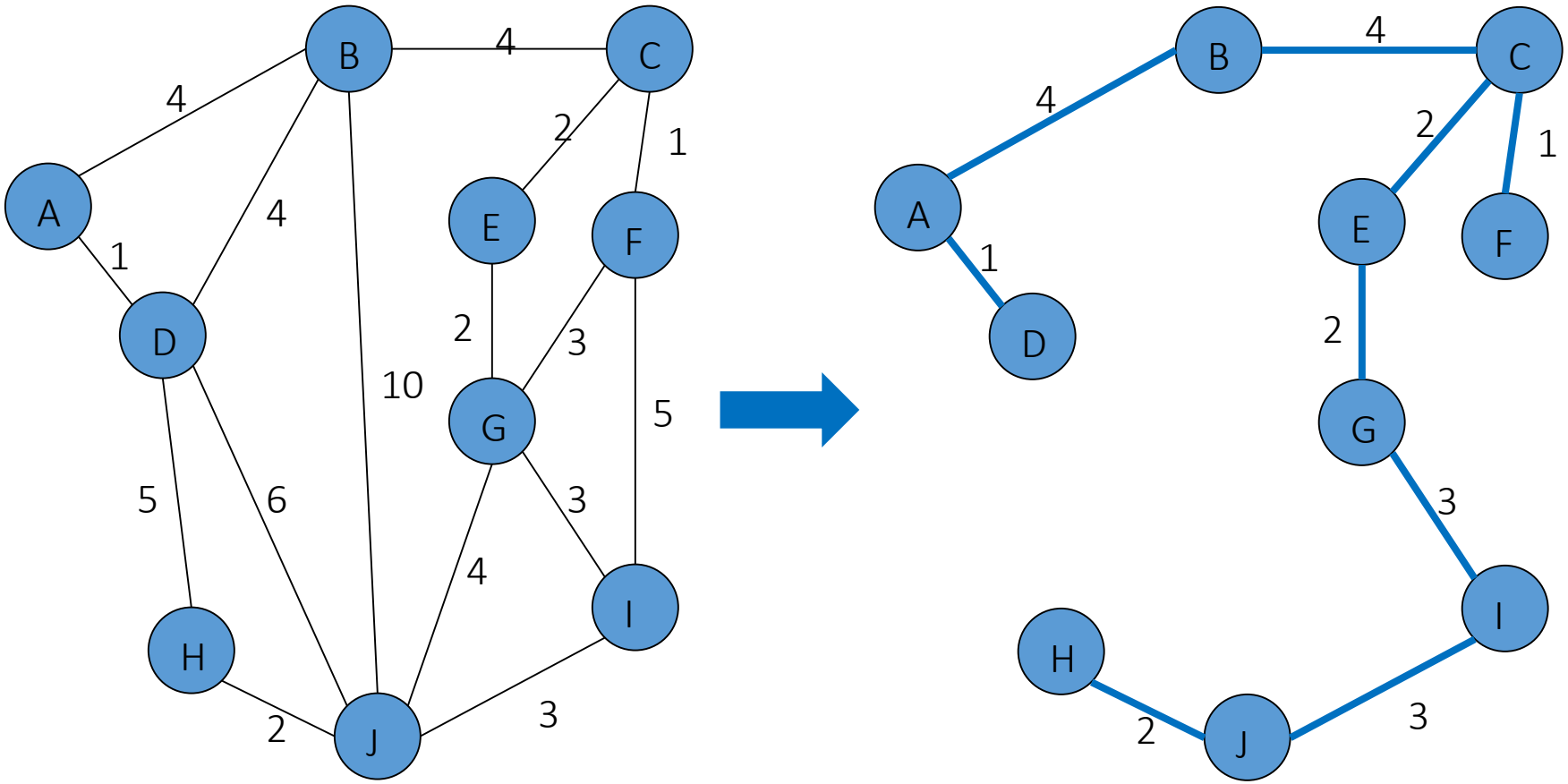
- Add Edge





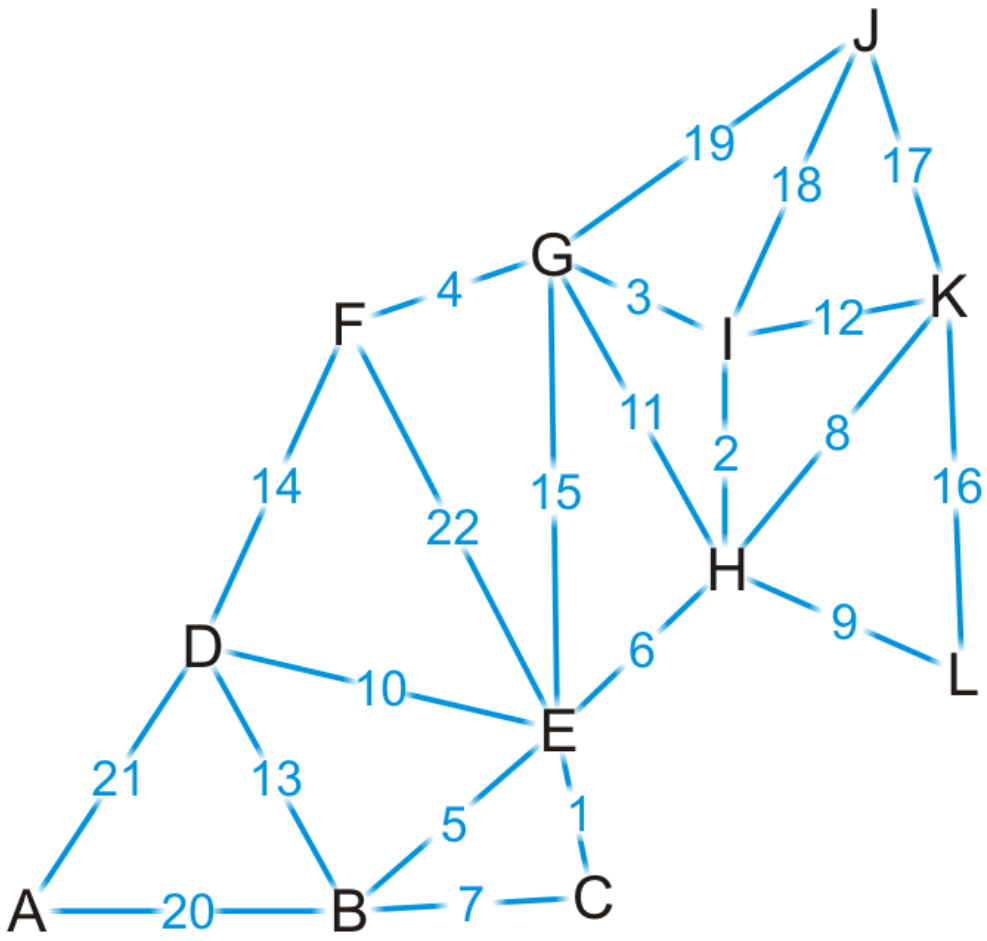
Kruskal's Algorithm – Example

- Minimum spanning tree



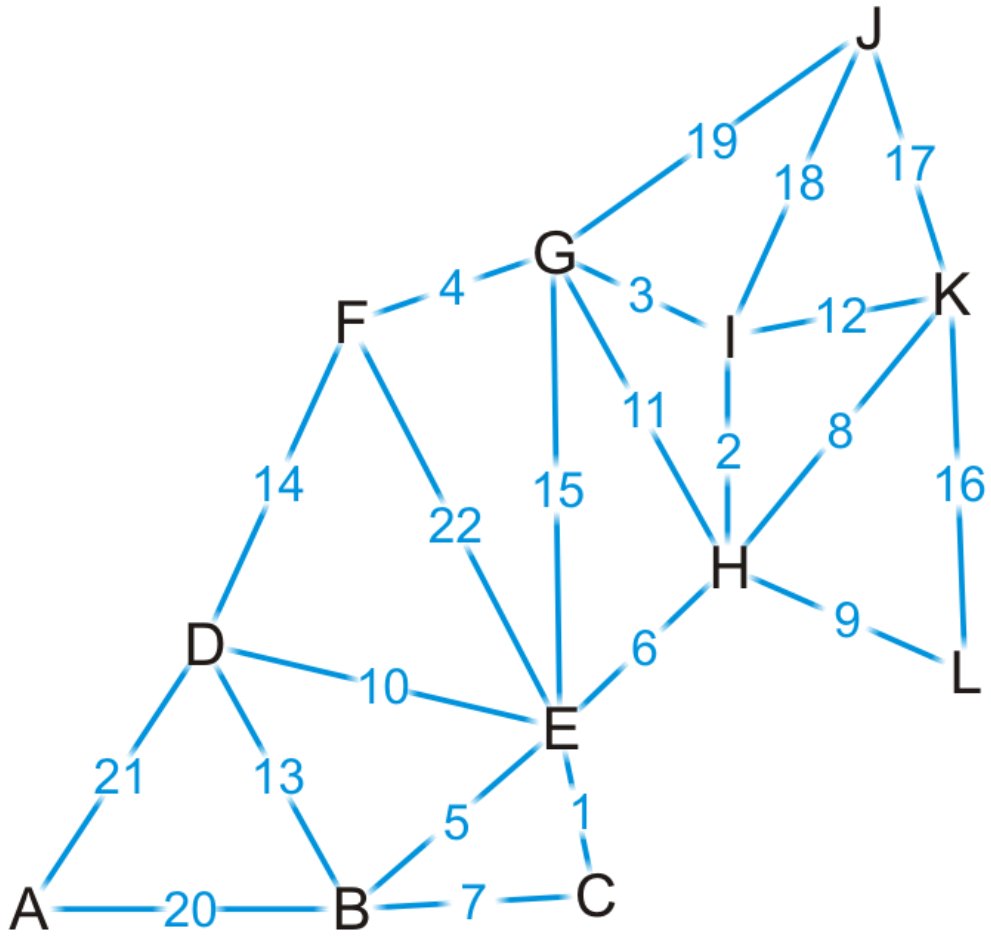
Kruskal's Algorithm – Example

- Complete graph



Kruskal's Algorithm – Example

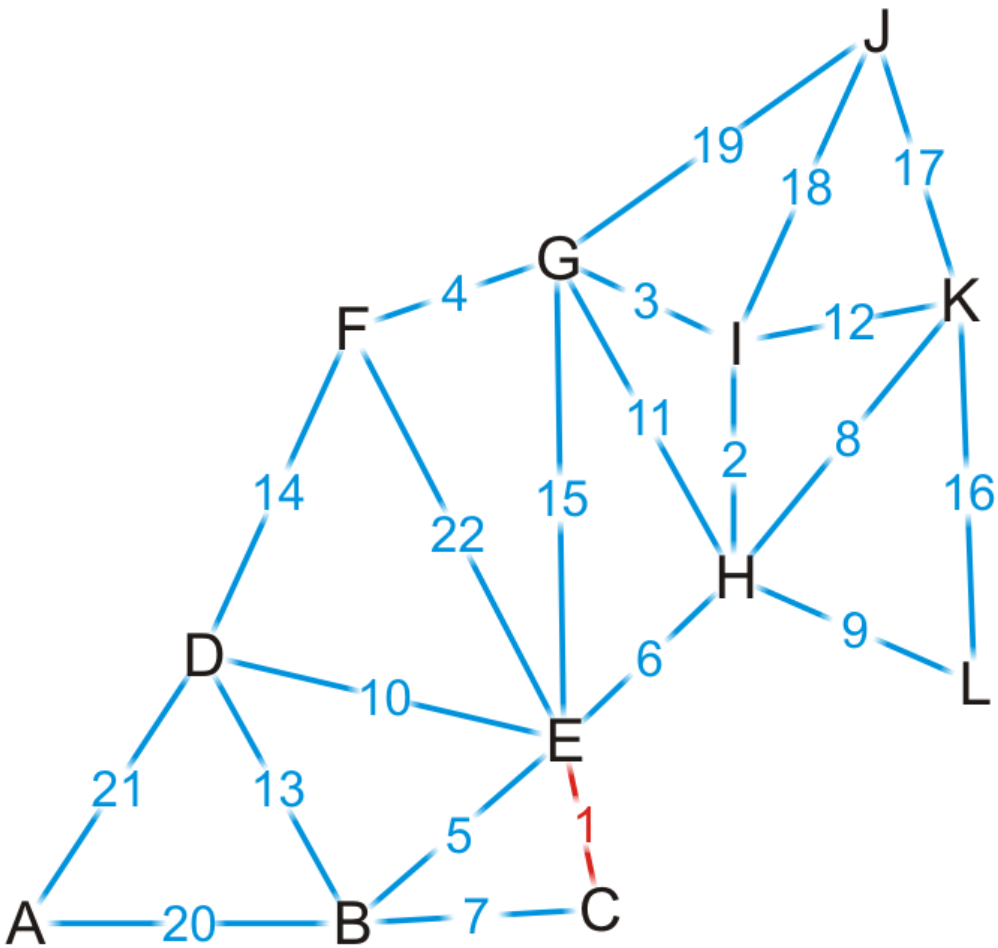
- Sort edges based on weight



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

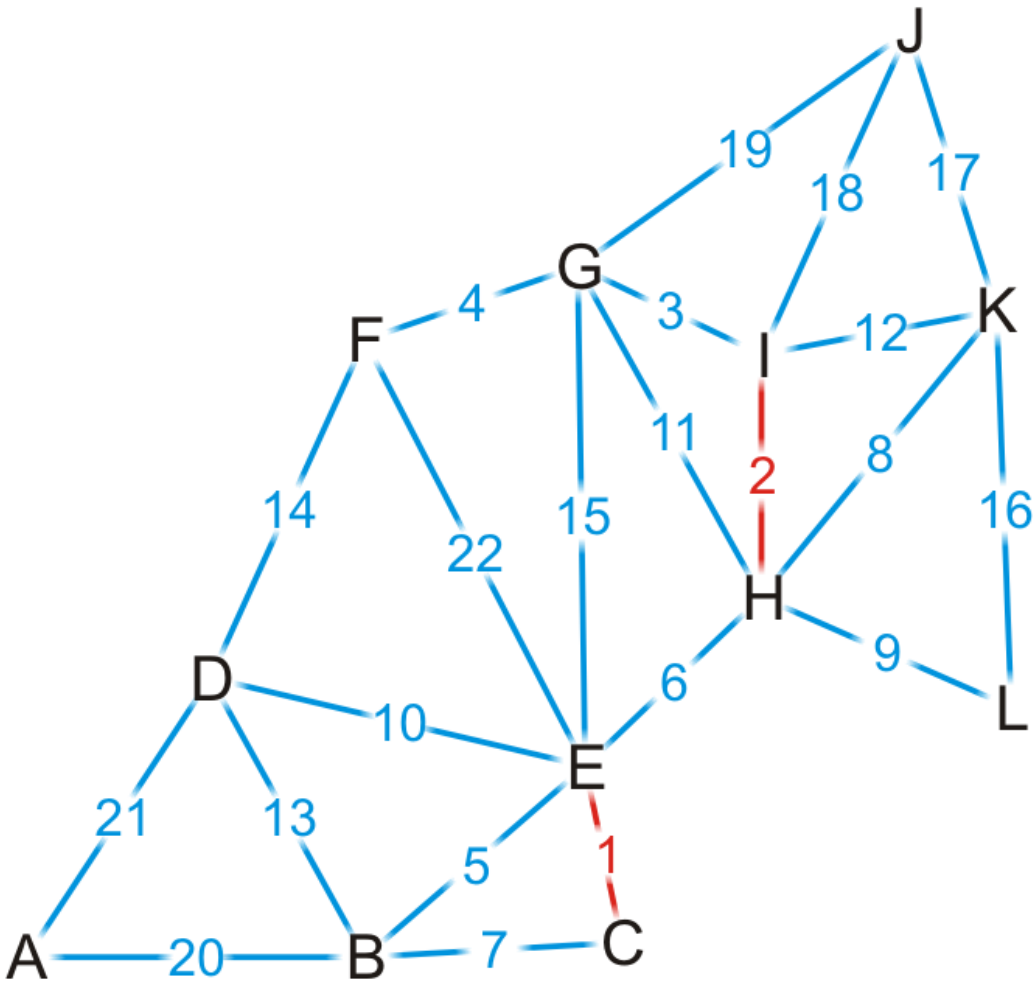
- We start by adding edge {C, E}



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

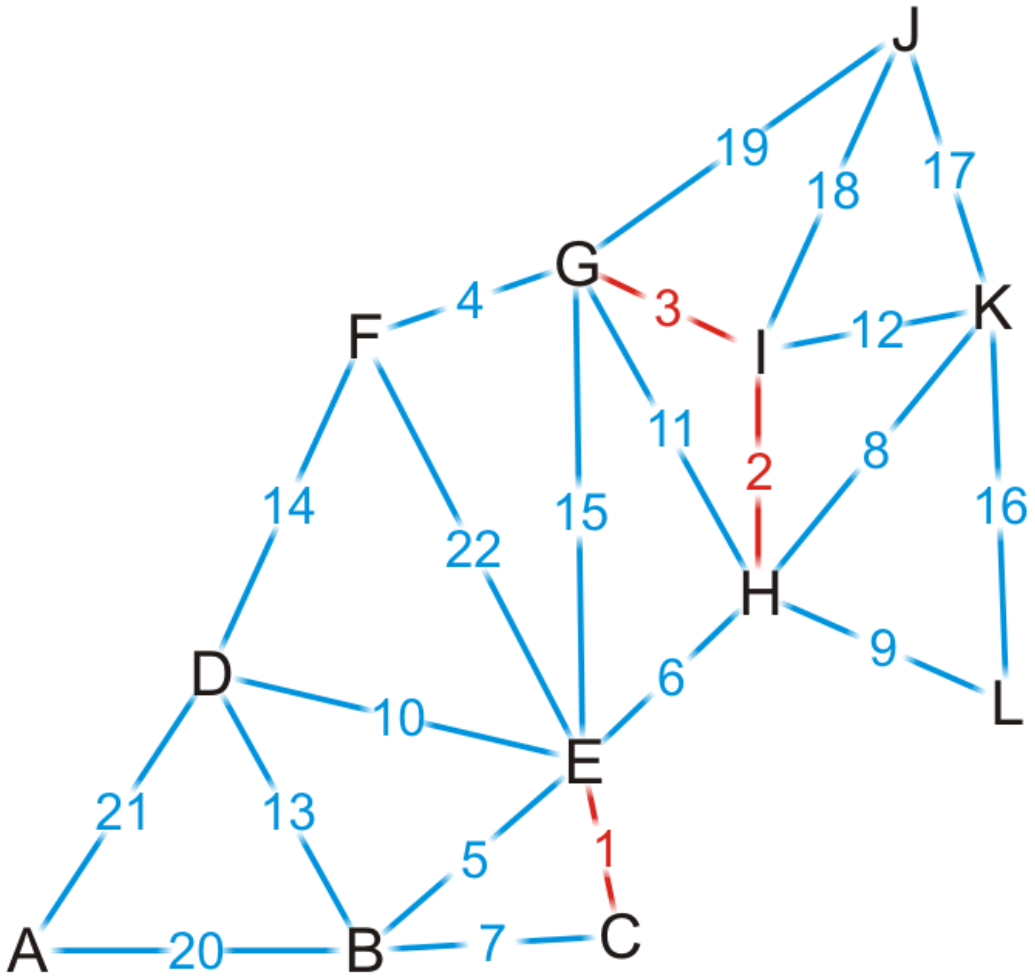
- We add edge {H, I}



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

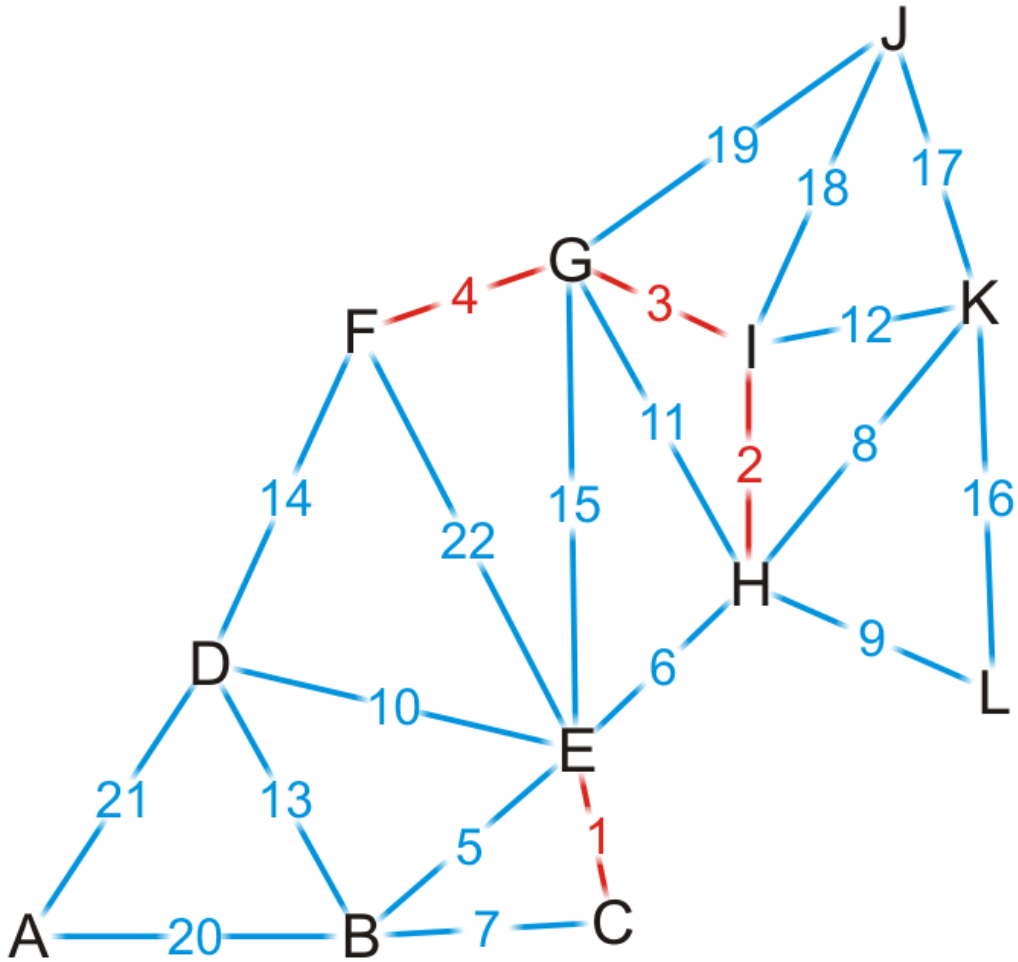
- We add edge {G, I}



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

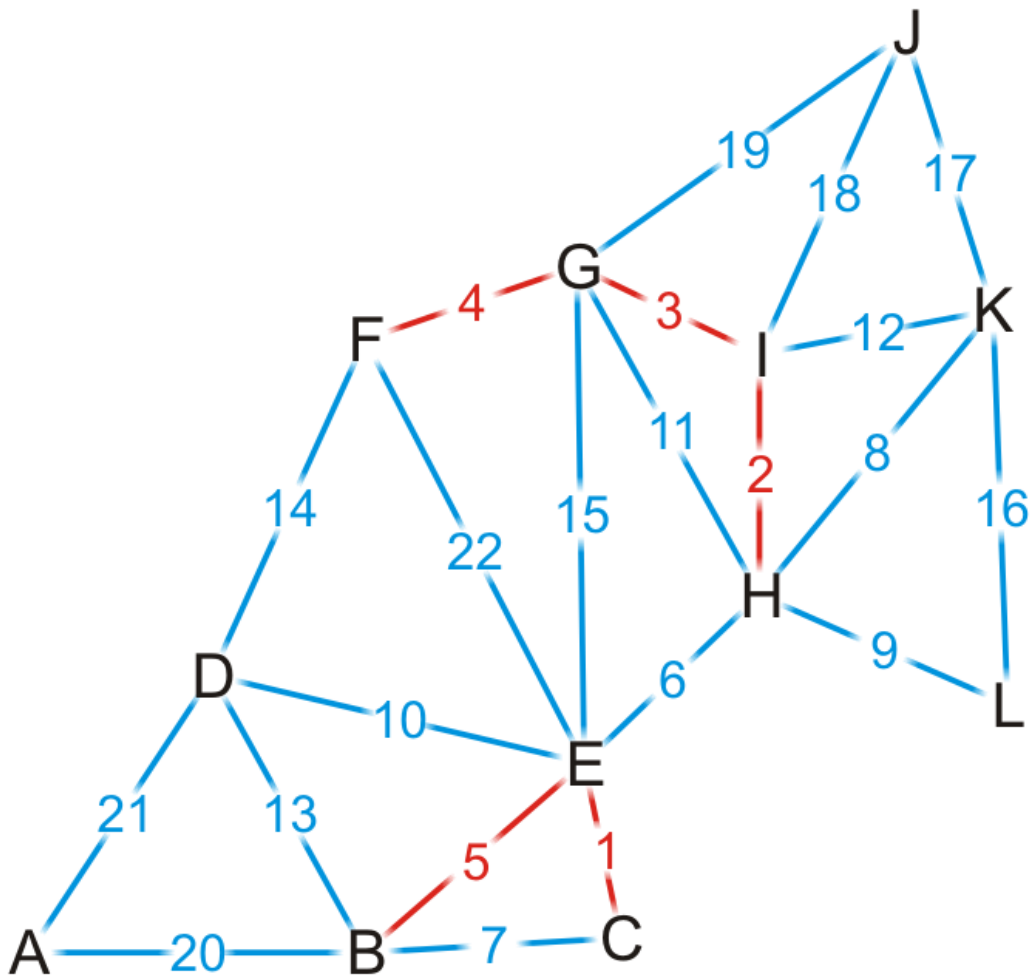
- We add edge {F, G}



{C, E}
{H, I}
{G, I}
→ {F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

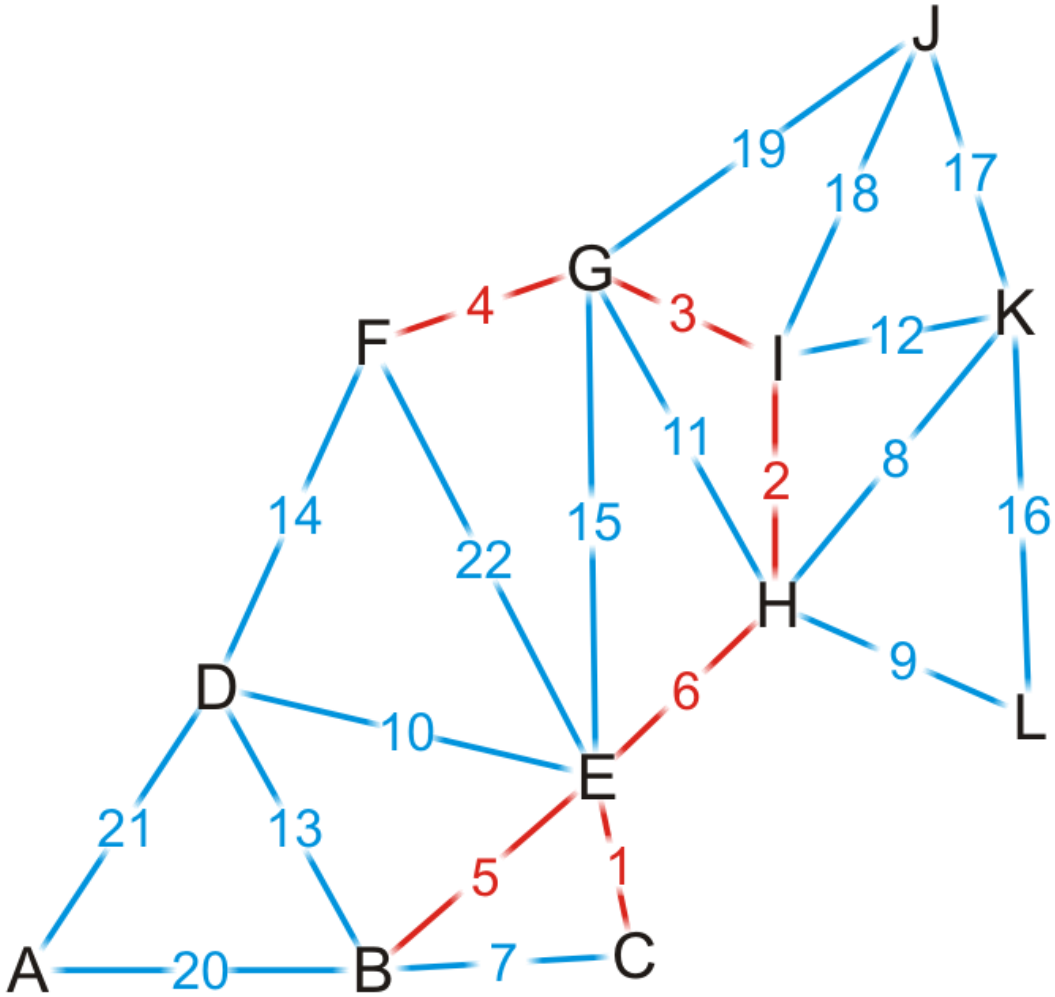
- We add edge {B, E}



{C, E}
{H, I}
{G, I}
{F, G}
→ {B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

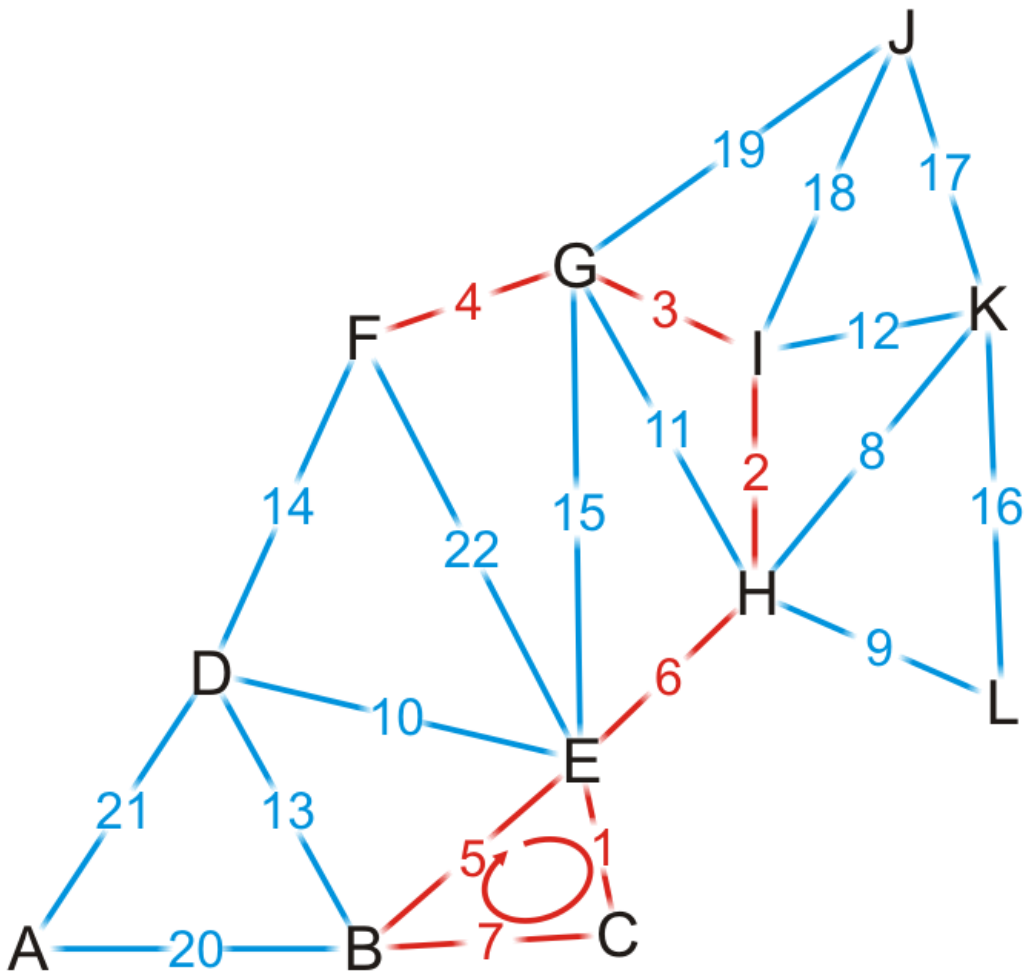
- We add edge {E, H}
  - This coalesces the two spanning sub-trees into one



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
→ {E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

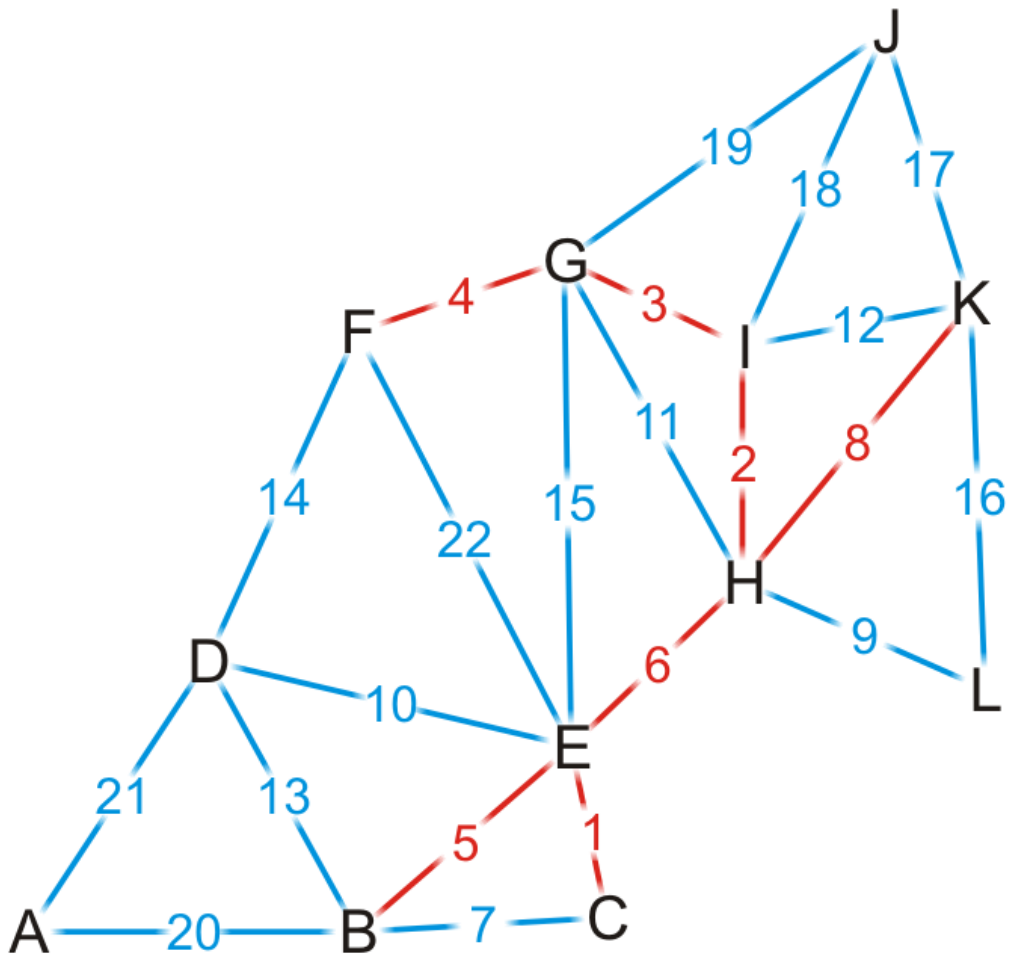
- We try adding {B, C}, but it creates a cycle



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
→ {B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

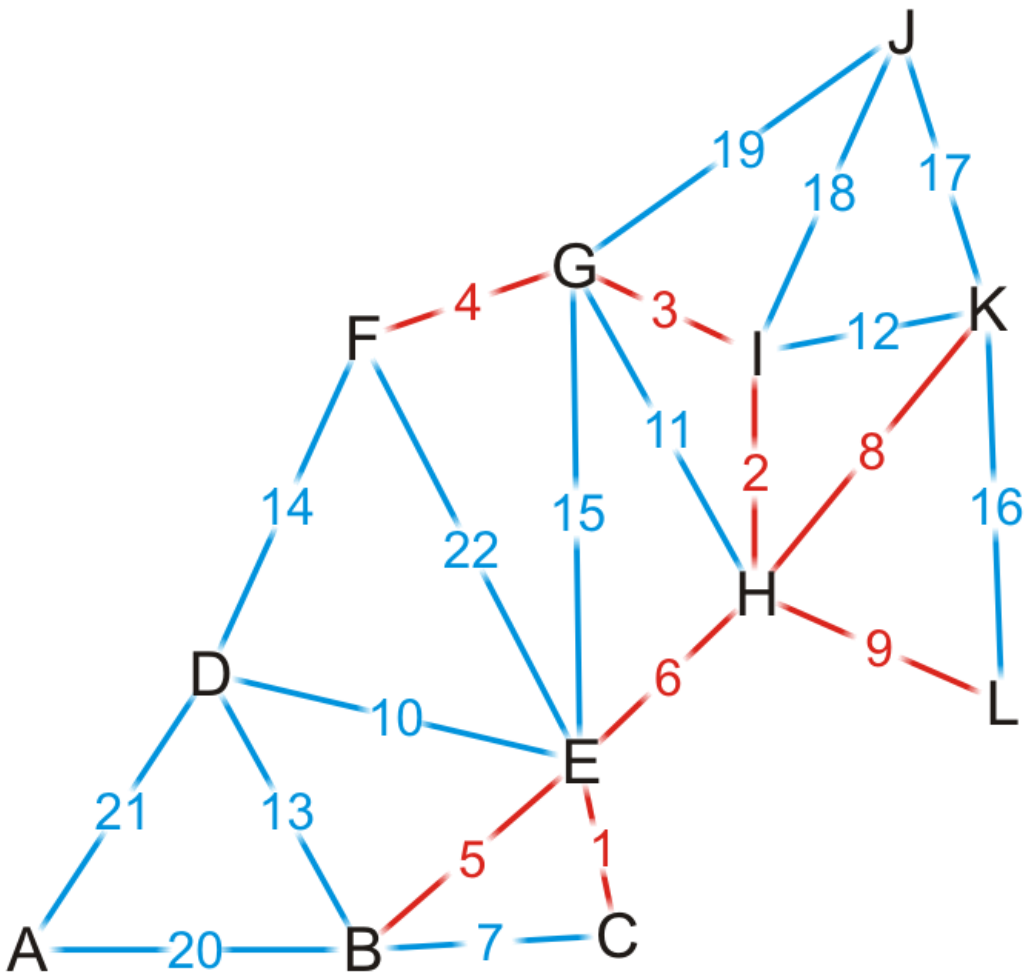
- We add edge {H, K}



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
→ {H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

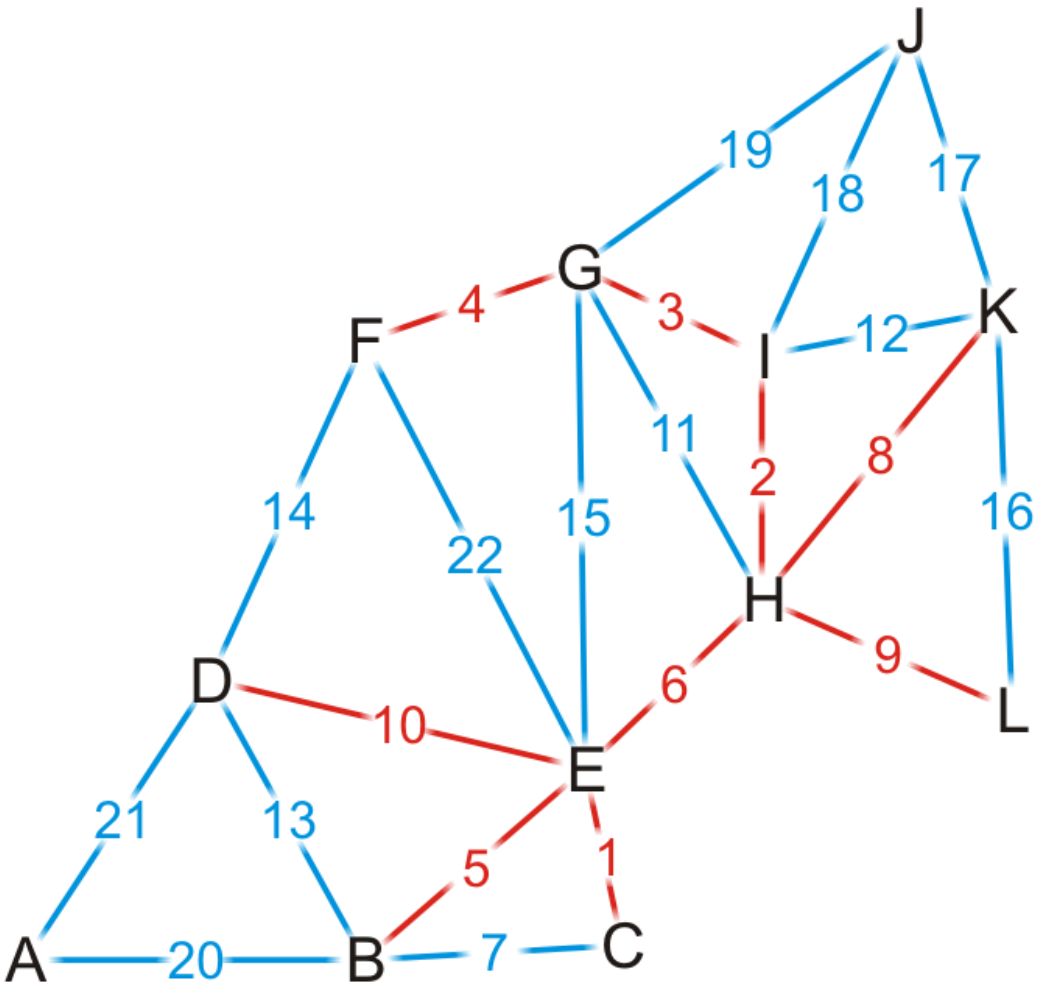
- We add edge {H, L}



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
→ {H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

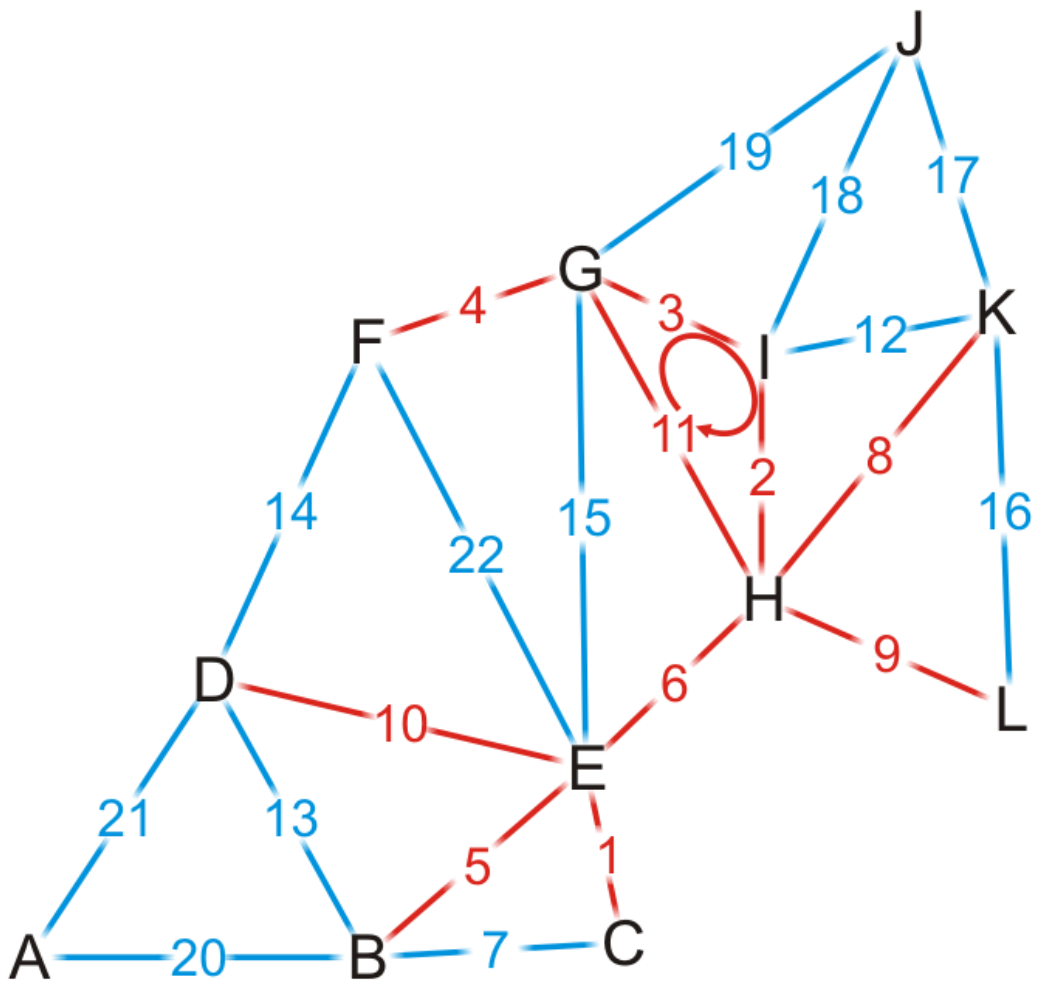
- We add edge {D, E}



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

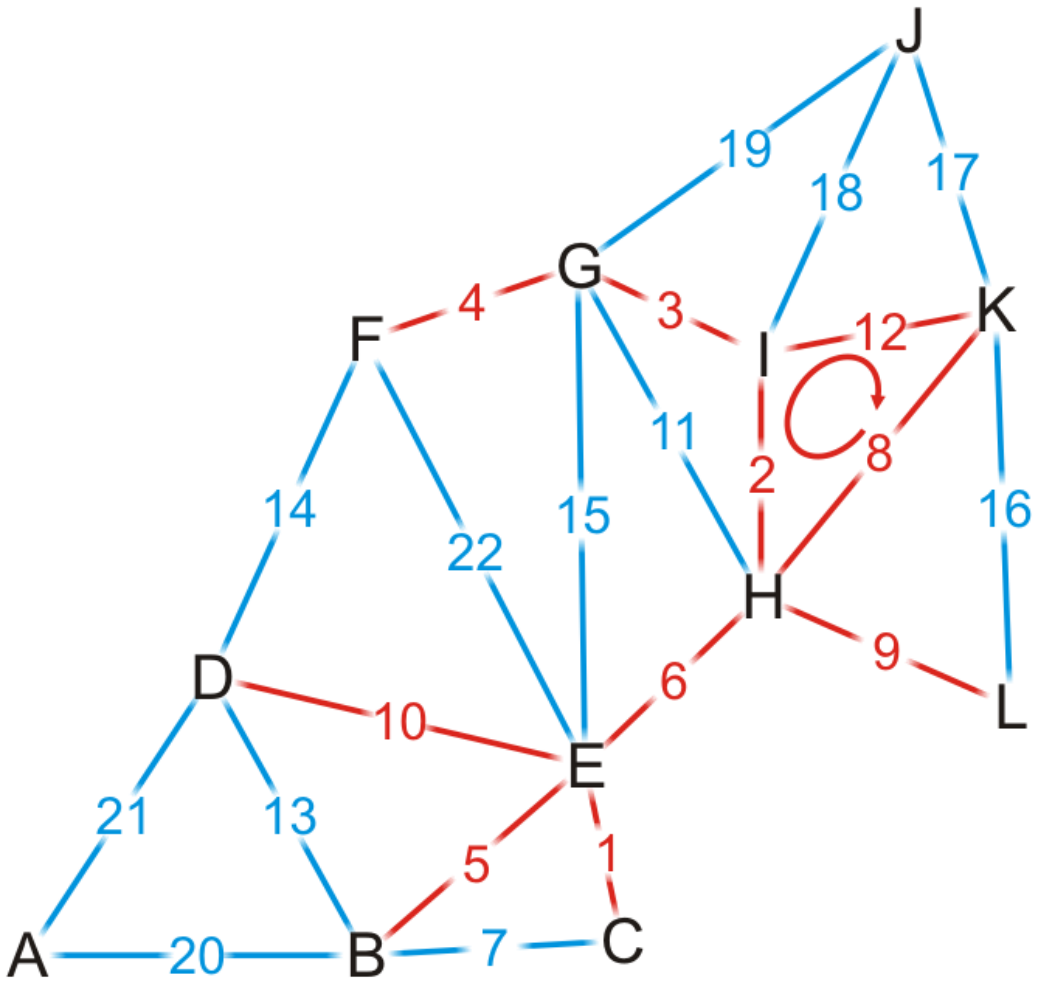
- We try adding {G, H}, but it creates a cycle



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
→ {G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

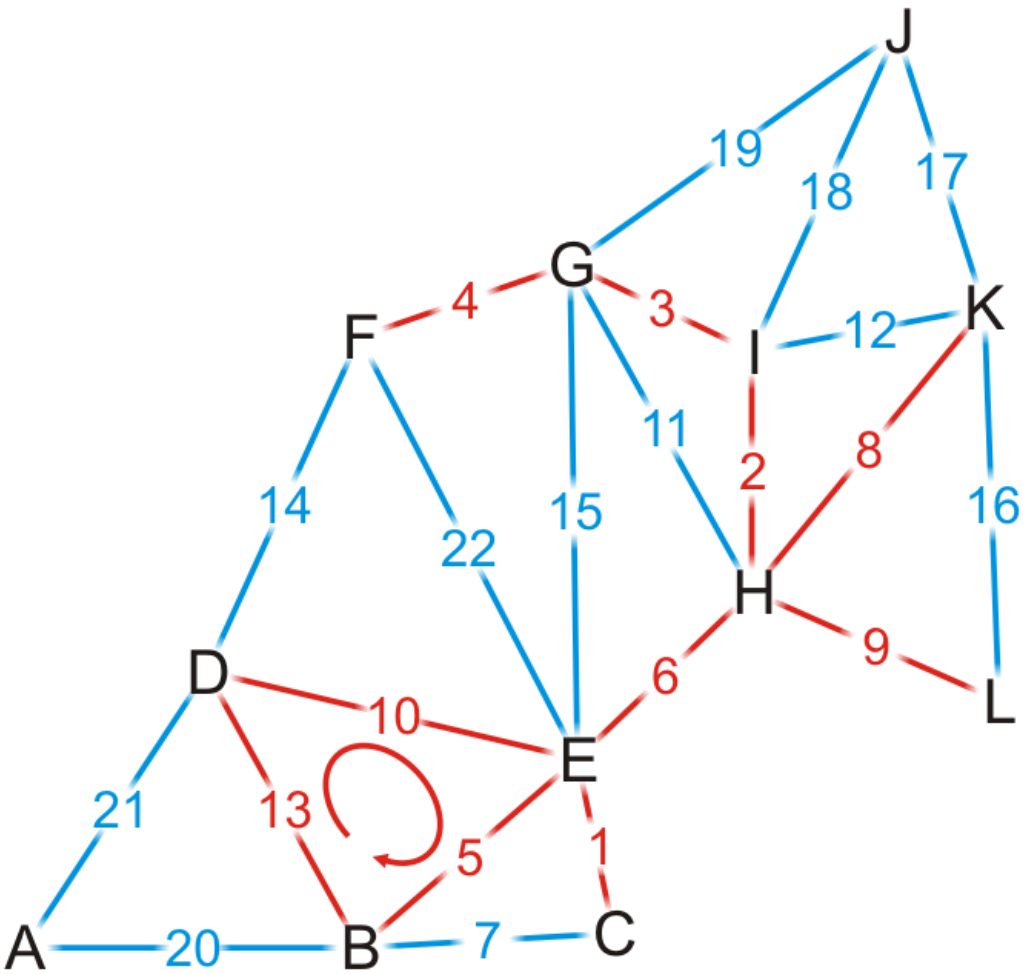
- We try adding {I, K}, but it creates a cycle



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
→ {I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

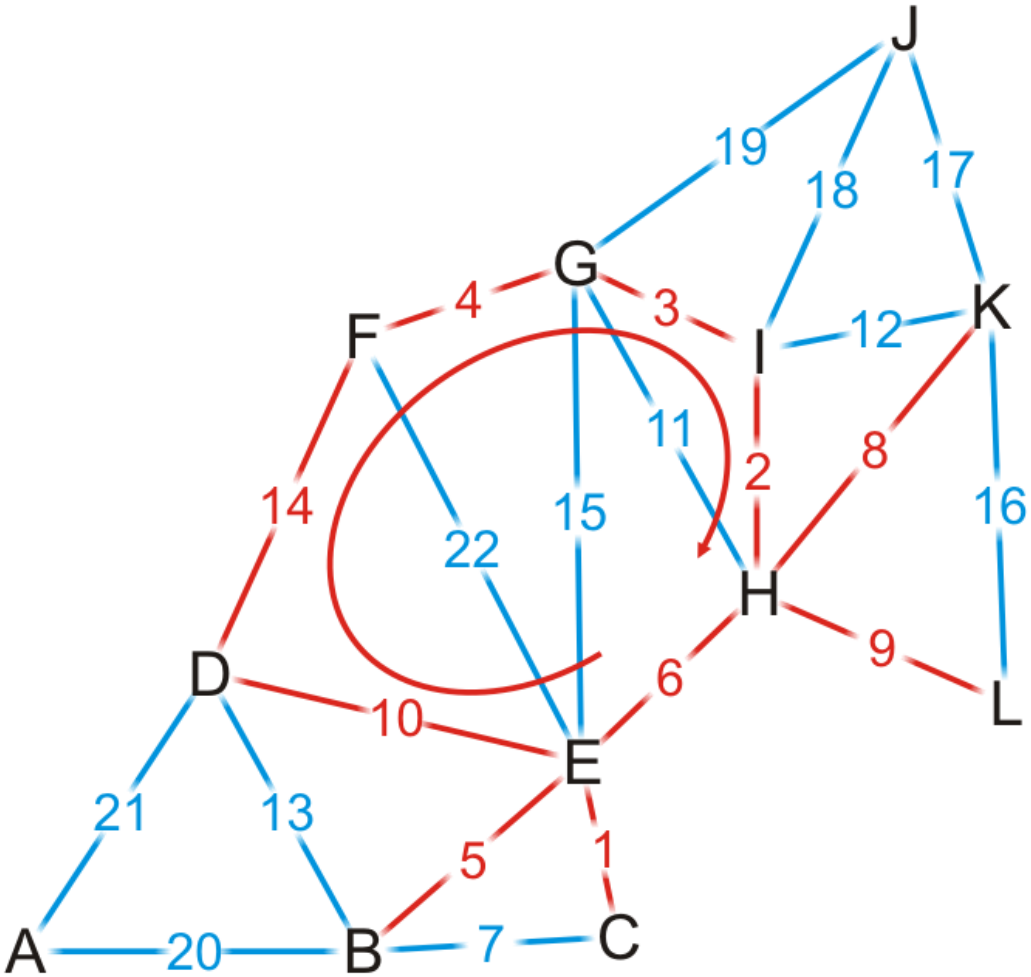
- We try adding {B, D}, but it creates a cycle



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
→ {B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

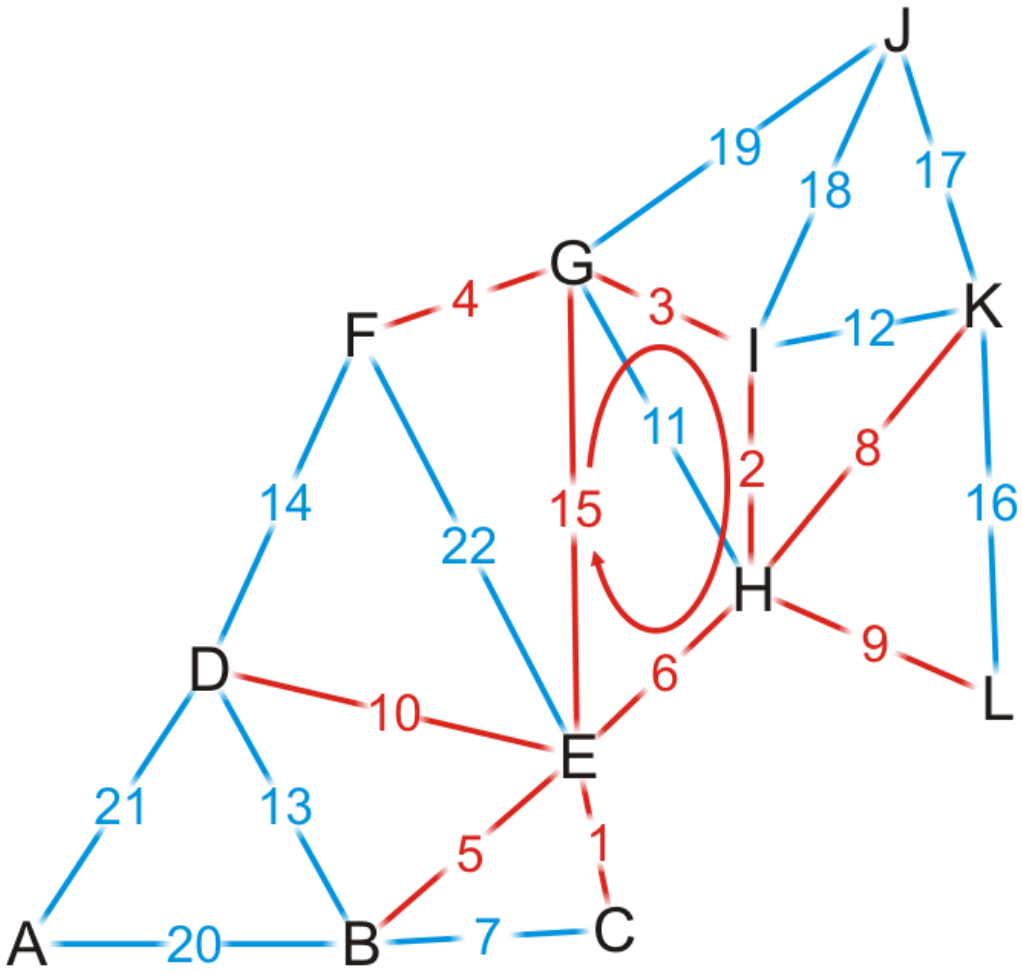
- ~~We try adding {D, F}, but it creates a cycle~~



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
→ {D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

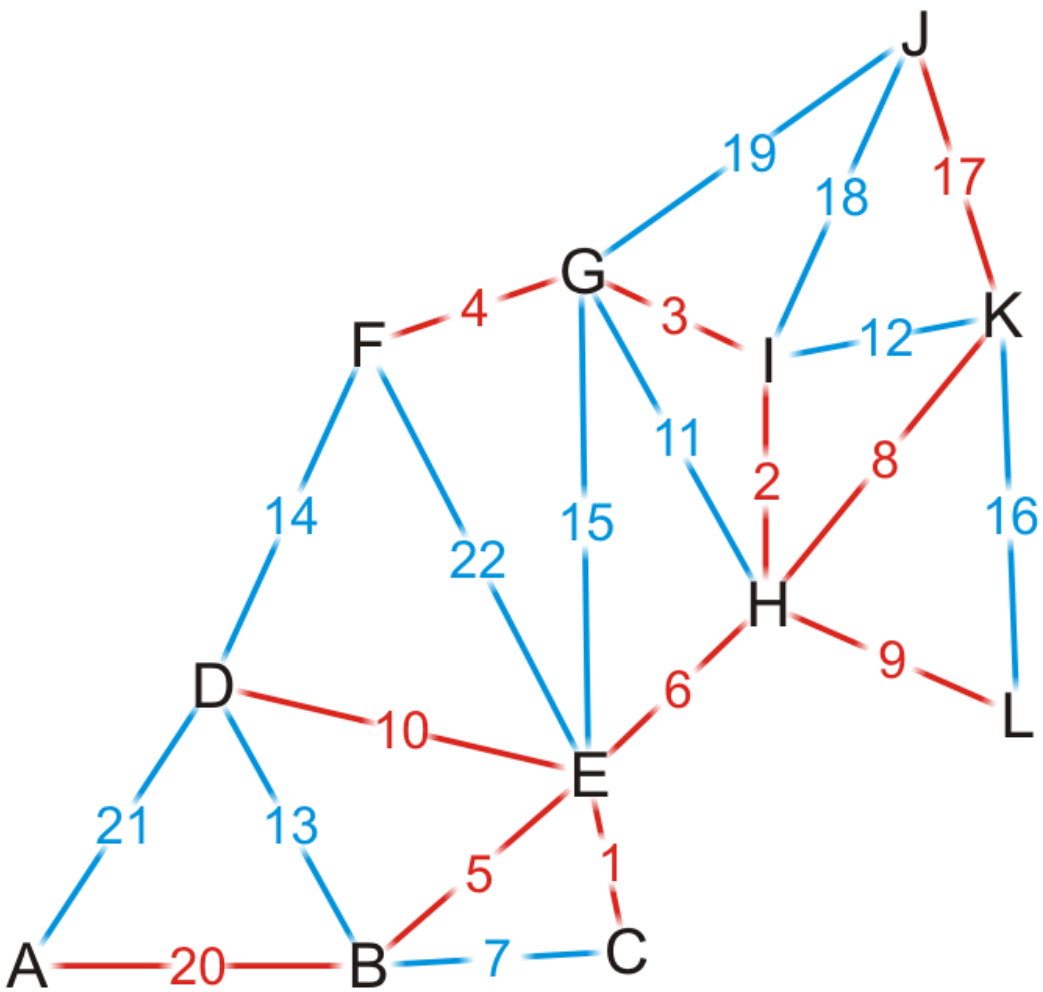
- We try adding {E, G}, but it creates a cycle



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
→ {E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

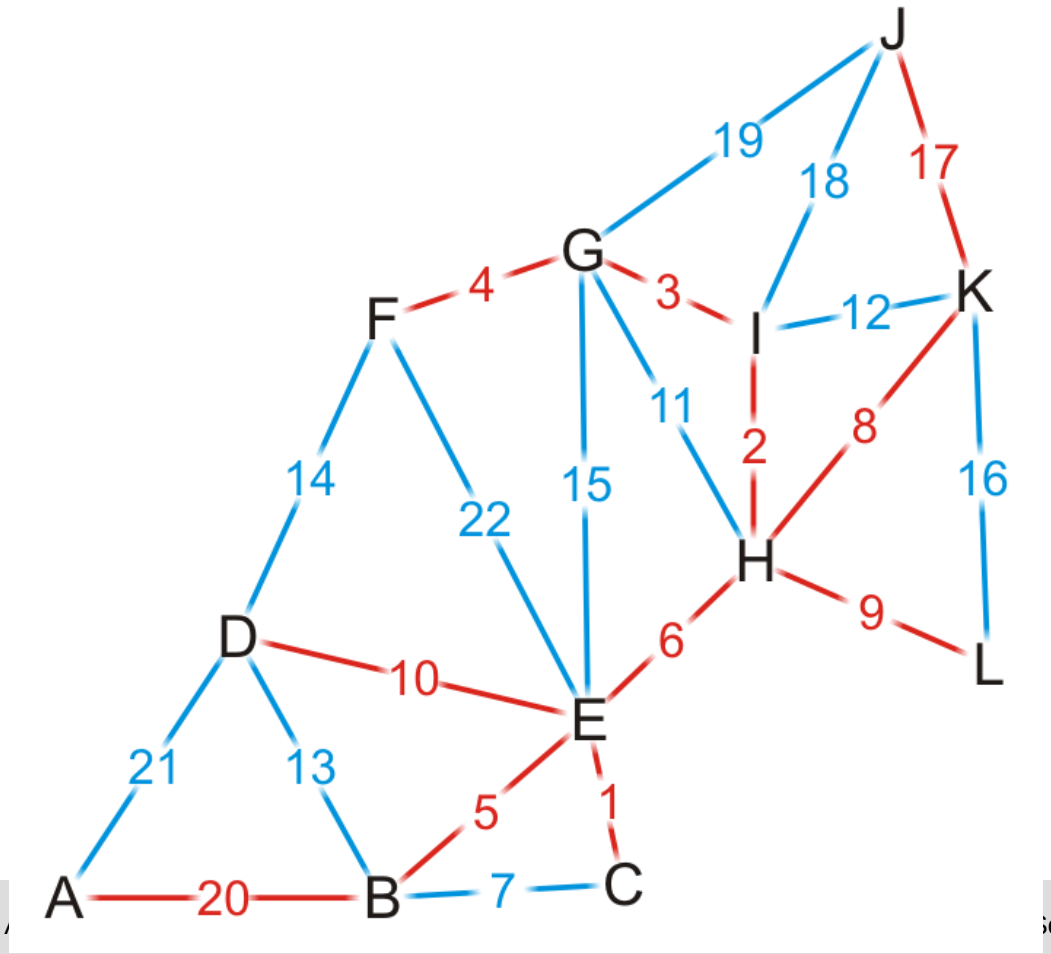
- By observation, we can still add edges {J, K} and {A, B}



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
→ {A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

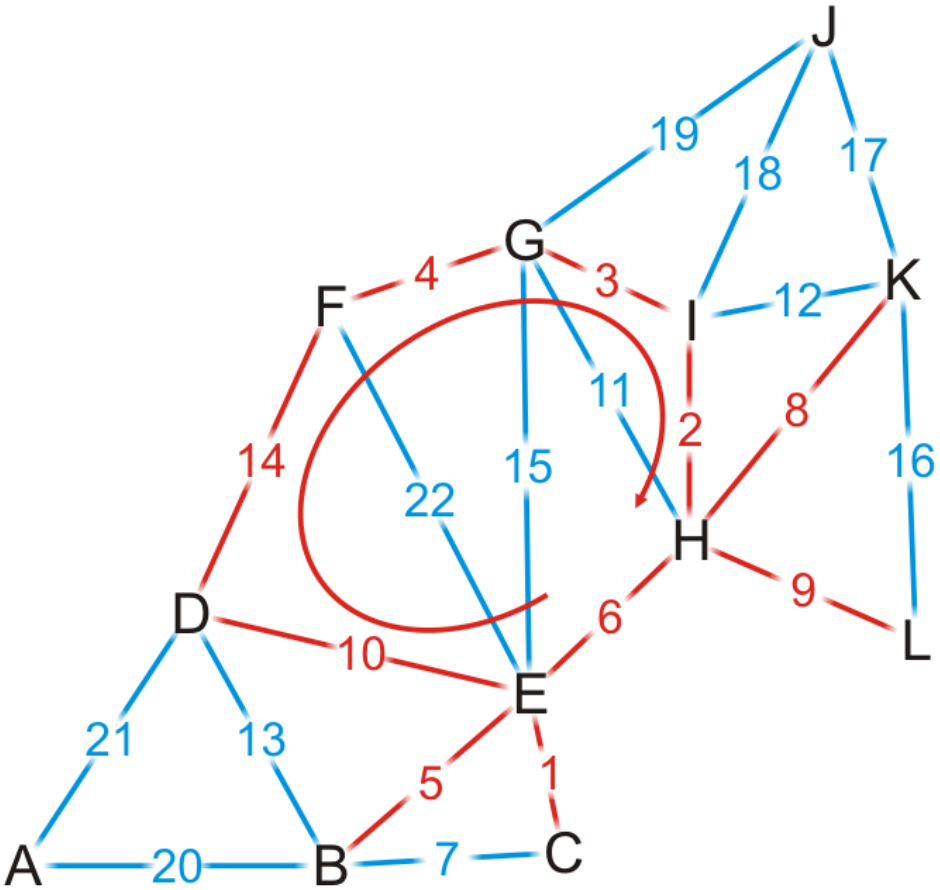
- Having added {A, B}, we now have 11 edges
  - We terminate the loop
  - We have our minimum spanning tree



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
→ {A, B}
{A, D}
{E, F}

Detecting a Cycle

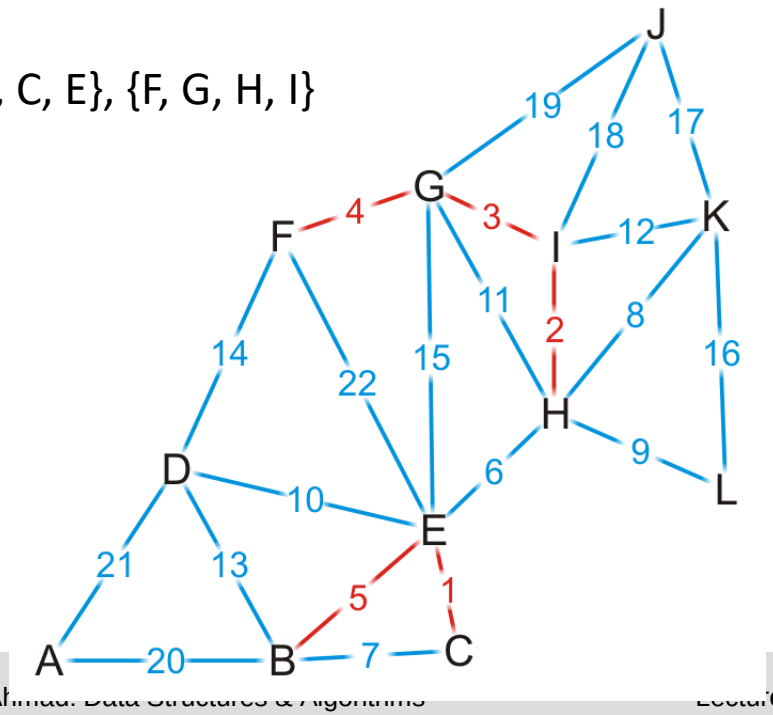
- To determine if a cycle is created, we could perform a traversal
  - A run-time of O(|V|)



Detecting a Cycle – Disjoint Sets

- Consider edges in the same connected sub-graph as forming a set

{B, C, E}, {F, G, H, I}

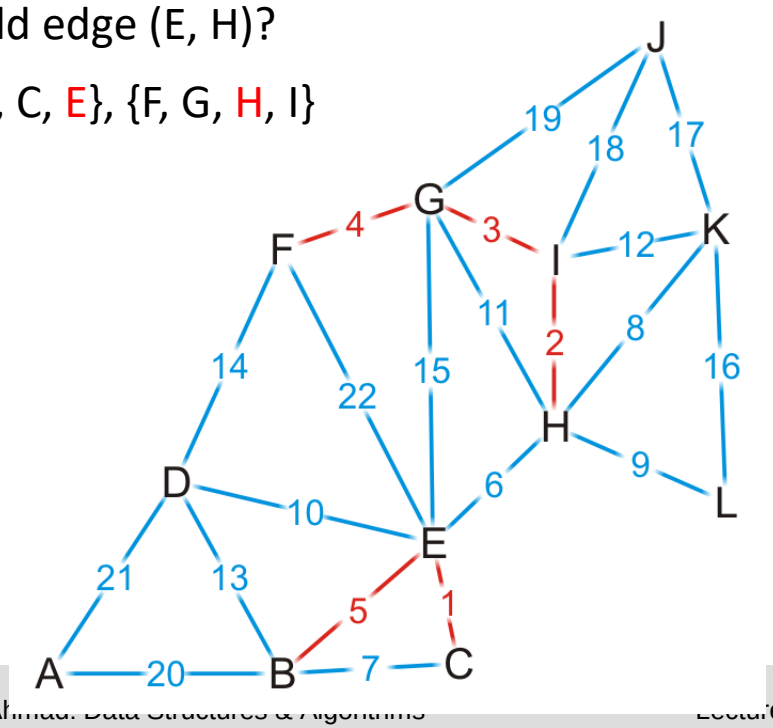


Detecting a Cycle – Disjoint Sets

- Consider edges in the same connected sub-graph as forming a set
- If the vertices of the next edge are in different sets
  - Take the union of the two sets

Add edge (E, H)?

{B, C, E}, {F, G, H, I}

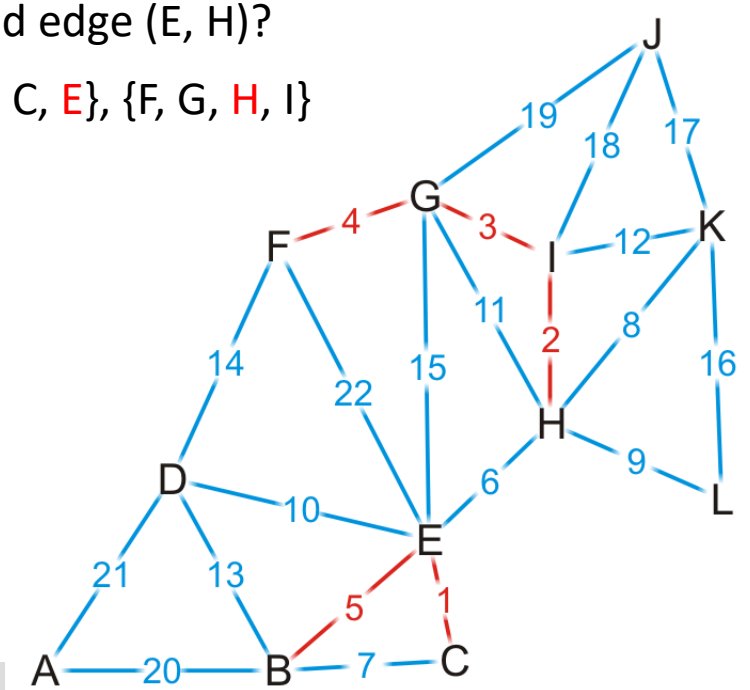


Detecting a Cycle – Disjoint Sets

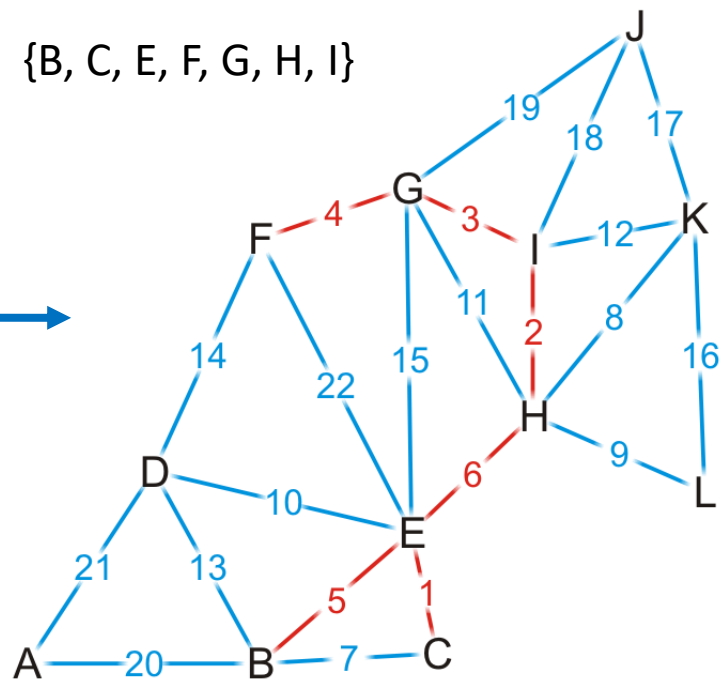
- Consider edges in the same connected sub-graph as forming a set
- If the vertices of the next edge are in different sets
  - Take the union of the two sets

Add edge (E, H)?

{B, C, E}, {F, G, H, I}



{B, C, E, F, G, H, I}

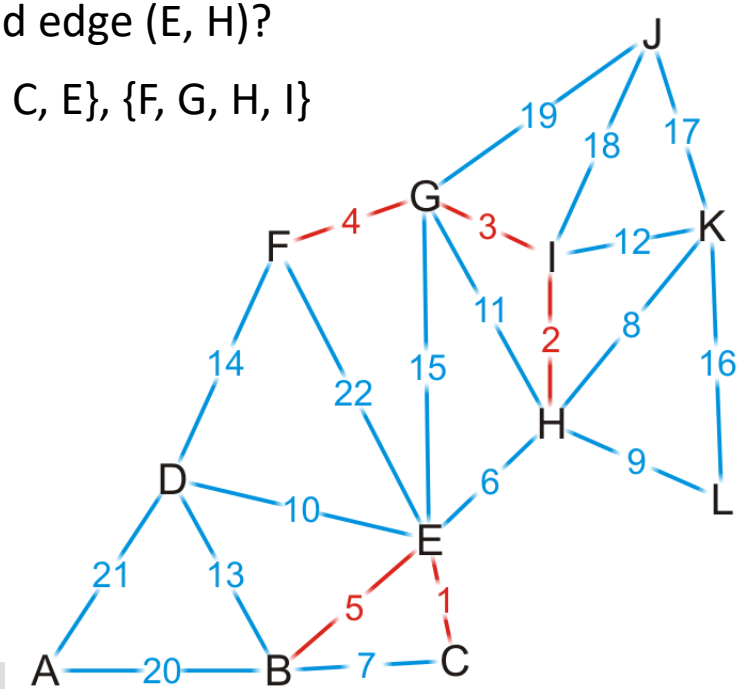


Detecting a Cycle – Disjoint Sets

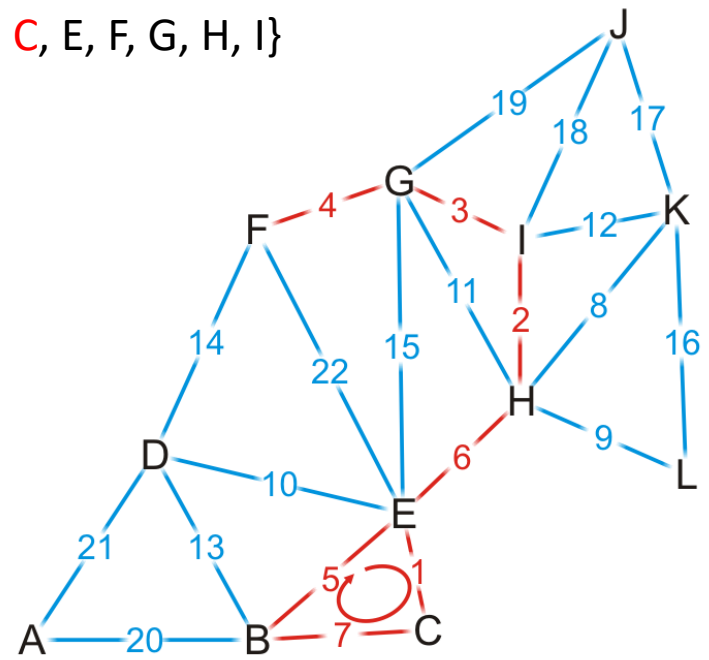
- Consider edges in the same connected sub-graph as forming a set
- If the vertices of the next edge are in different sets
  - Take the union of the two sets
- Do not add an edge if both vertices are in the same set

Add edge (E, H)?

{B, C, E}, {F, G, H, I}



{B, C, E, F, G, H, I}



KRUSKAL(V, E, w)

```
A ← ∅
for each vertex v ∈ V
        do MAKE-SET(v)                                  } O(V)
sort E into non-decreasing order by w                   } O(ElgE)
for each (u, v) taken from the sorted list              ← O(E)
        do if FIND-SET(u) ≠ FIND-SET(v)
                then A ← A ∪ {(u, v)}
                        UNION(u, v)                     ← O(lgV)
return A
```

Running time: O(V+ElgE+ElgV)=O(ElgE) – dependent on the implementation of the disjoint-set data structure

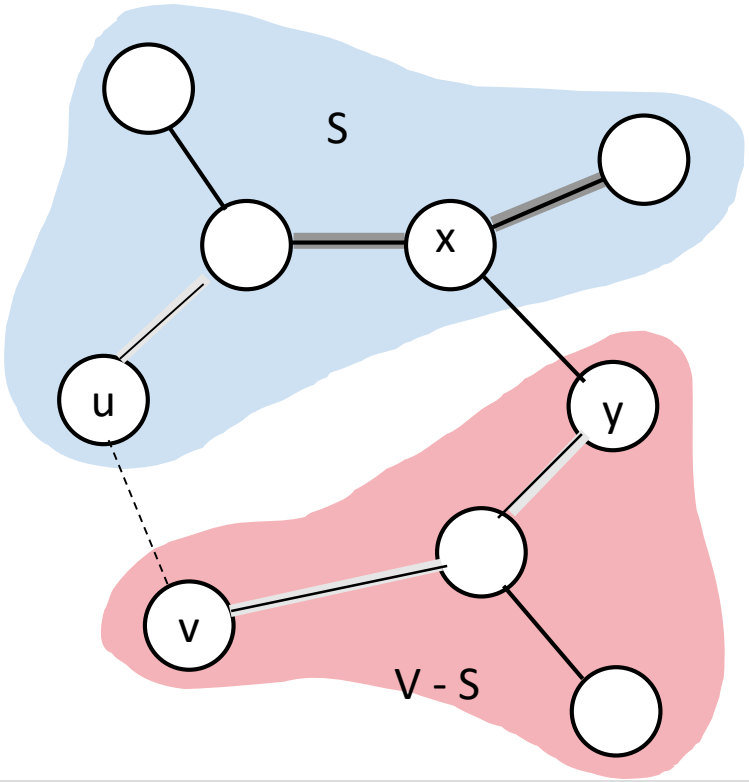
KRUSKAL(V, E, w) (cont.)

```
A ← ∅
for each vertex v ∈ V
        do MAKE-SET(v)                                  } O(V)
sort E into non-decreasing order by w                   } O(ElgE)
for each (u, v) taken from the sorted list              ← O(E)
        do if FIND-SET(u) ≠ FIND-SET(v)
                then A ← A ∪ {(u, v)}
                        UNION(u, v)                     ← O(lgV)
return A
```

- Running time: O(V+ElgE+ElgV)=O(ElgE)
- Since E=O(V^2), we have lgE=O(2lgV)=O(lgV) } O(ElgV)

Kruskal's Algorithm

- Kruskal's algorithm is a **"greedy"** algorithm
- Kruskal's greedy strategy produces a globally optimum solution
- Proof for generic approach applies to Kruskal's algorithm too



Kruskal’s Algorithm vs Prim’s Algorithm

- In **prim’s algorithm**, graph must be a **connected**
- **Kruskal’s** algorithm can function on **disconnected** graphs too
- **Prim's algorithm** is significantly faster for **dense graphs** with more number of edges than vertices
- **Kruskal's algorithm** runs faster in the case of **sparse graphs**